EmbodiedEvo: Self-improving Coding Agents for Physical Intelligence

Wenyi Li[†], **Yizhe Chi, Youjie Zheng, Chunyu Liu, Xiaoqiu Wang, Mingju Gao, Zheng Jiang, Deyao Hong, Calvin Xiao, Kaisen Yang**[‡], **Qinhuai Na**[‡]

Navers Lab, Einsia.AI | **Tsinghua University**

[†]Project Lead [‡]Corresponding Authors

Bringing general-purpose agents into the physical world calls for an executable representation of physical intelligence. Compared with vision-language-action (VLA) models, which map observations and language instructions directly to actions, code makes control logic explicit and reusable. Code can serve not only as a policy representation but also as the workspace in which the agent solves a task. Rather than following a prescribed workflow of planning, execution, and verification, the agent can autonomously inspect the state, revise and execute code, and use feedback to choose the next step. The success of this SWE-style loop in digital environments motivates its extension to physical tasks. Yet physical execution remains prone to recurring failures. To retain and refine experience for later scenes, we introduce **EmbodiedEvo**, where a coding agent evolves reusable Skills and Lessons through feedback on a single Playground scene, then freezes the resulting harness for diverse new scenes. On RoboDojo-Sim, EmbodiedEvo achieves an average score of 44.48 and a success rate of 37.82%, surpassing all officially verified results. Paired tests show the largest gains on memory-dependent tasks and successful reuse of coordinated procedures, while fine contact control remains difficult. Experience from stronger models also improves weaker executors: Astra's experience raises Sol and Luna's success rates by 8.86 and 5.50 percentage points over no experience. Compared with raw interaction history, Skills and Lessons achieve comparable task performance at 53.0% lower execution cost.

Date: October 13, 2026
Homepage: https://lab.einsia.ai/embodiedevo
Correspondence: liwenyi@einsia.ai, kaisenyang@einsia.ai, nana@einsia.ai

Einsia



Figure 1 Overview of EmbodiedEvo. During task-specific Playground interaction, a coding agent develops control programs and revises reusable Skills and Lessons through physical feedback. The resulting harness is frozen and inherited by fresh Test agents on new scenes.

1 Introduction

"Intelligence is determined by the dynamics of interaction with the world."

— Rodney A. Brooks

Agents are beginning to act beyond the digital world, entering real-world physical environments. Large language models have enabled agents to reason, use tools, and iteratively develop solutions through feedback in software environments [1]. Extending this autonomy to physical tasks requires agents to connect reasoning with perception and execution while adapting to the consequences of their actions. Bringing these capabilities to robotics calls for a representation of physical knowledge that agents can execute, inspect, and refine.

Compared with vision-language-action (VLA) models, which map observations and language instructions directly to robot actions [2, 3], code makes control logic explicit, editable, and reusable. Code can represent task-relevant objects, estimated states, relations, and progress in a form that the agent can inspect and update. We define this as "*Code as World*". Code also provides a workspace for developing solutions. Early code-as-policy systems synthesize programs within predefined generation and execution scaffolds [4]. Software-engineering-style (SWE-style) agents extend this process by directing their own inspection, editing, execution, and diagnosis [1]. Applied to robotics [5, 6], this autonomy lets the agent decide when to observe, which code to execute, and how to verify outcomes and recover from failure. We define this agent-directed interaction as "*Agent as Policy*".

Reliable physical execution requires more than editable code: programs must be grounded in a robot's observations, action conventions, and physical constraints. Even GPT-6 Astra can fail through inaccurate approach geometry, mistimed corrections, and poor coordination between arms [7]. Interaction reveals useful motions, tolerances, and checks, but knowledge confined to a single attempt must be rediscovered in later scenes. Our baseline traces (Section 3.3) identify recurring failures that call for reusable progress checks, physical postconditions, and recovery procedures. These observations motivate evolving the harness so that subsequent agents can build on prior interaction. Building on feedback-driven skill optimization and harness refinement [8–10], *harness evolution* retains and revises control code and experience while keeping model weights and the robot interface fixed.

To this end, we introduce **EmbodiedEvo**, a self-improving coding agent that implements harness evolution through reusable Skills and Lessons. We build a Playground for the agent using a single scene from each task. In this scene, the agent executes code in a persistent simulation session and iteratively refines its Skills and Lessons through physical feedback. Skills store control procedures and their usage conditions, while Lessons retain observed failure modes, constraints, and recovery experience. We then freeze the resulting harness and supply it to independent Test agents across a more diverse set of layouts of the same task, all distinct from the Playground layout. Each agent starts with a fresh workspace, Python state, and simulator, using the read-only harness to develop local programs while continuing to observe, execute, and recover.

Experiments on RoboDojo [7] simulation tasks show that inherited experience improves subsequent execution. With task descriptions and wiki recipes, Astra/xhigh reaches 37.82% Average success rate over 50 Test scenes per task, compared with the published Astra result of 22.48% . Paired tests hold the executor and scenes fixed, raising success from 8.50% to 30.17% with task descriptions alone. The largest gains occur on memory-dependent tasks, alongside successful reuse of coordinated procedures such as block stacking; fine insertion and liquid handling remain difficult. Experience also transfers across models: Astra's Skills and Lessons raise Sol and Luna's success rates by 8.86 and 5.50 percentage points over no experience. These results distinguish the ability to produce useful control knowledge from the ability to apply it.

Our contributions are:

- **Physical self-improvement through an evolving harness.** We introduce EmbodiedEvo, which turns physical interaction into persistent control procedures and decision guidance. *Code as World, Agent as Policy* makes this knowledge explicit, editable, and reusable while model weights and the native robot interface remain fixed.
- **Experience that couples actions with their conditions.** Our harness combines executable Skills with Lessons on applicability, failure modes, and outcome verification. A Playground-to-Test protocol acquires

this knowledge on one scene per task and freezes it for independent agents, enabling reuse across unseen layouts and different models without additional Playground exploration.

- **Acquiring and applying experience are distinct capabilities.** We show that unfinished tasks can yield useful knowledge and that weaker executors benefit from experience they struggle to acquire themselves. EmbodiedEvo surpasses all officially verified RoboDojo simulation results in Average Score and success rate; Skills/Lessons retain comparable performance to raw history at 53.0% lower Test cost.

2 Related Work

General-Purpose Models for Robot Control. VLA models integrate semantic knowledge, embodied reasoning, heterogeneous data, and interaction experience into learned policies [2, 3, 11, 12]. RoboDawn uses in-context demonstrations to ground robot-interface use, while early Astra evaluations expose geometric and coordination failures in direct control [7, 13]. Coding approaches make behavior editable: CaP-X and EmbodiedSWE develop programs through interaction, RAPID adapts demonstration-derived programs to scene geometry, and closed-loop software optimization uses archived implementations to acquire policies [5, 6, 14, 15]. HexaAnything combines coded state and policies with perception and control tools [16]. EmbodiedEvo combines this program-level adaptability with reusable experience, so that the agent can draw on earlier interactions while revising code for a new scene.

Self-Evolving Agents and Harnesses. Self-improvement spans policy learning in R-Zero and SkillRL [17, 18], and revision of skills, memory implementations, and context-construction code in SkillOpt, MemEvolve, and SHAPER [8, 19, 20]. Continual Harness and RegenHarness study harness refinement and controlled revision [9, 21]; Hyperagents revises task and meta-agent programs, while EnvHarness customizes the surrounding environment [22, 23]. CORAL supports asynchronous exploration with shared notes and skills [10]. Our focus is the physical control knowledge stored in the harness: executable procedures and lessons about when they work, why they fail, and how to recover.

Reusable Physical Experience. RATs acquires and freezes code skills before downstream tasks arrive, including transfer across coding agents [24]. RoboSkill combines reusable code and textual guidance with tactile feedback, while ASPIRE evolves skill libraries across task sequences [25, 26]. RoboHarn-Evo updates hierarchical physical knowledge, EmbodiedRSI selects experiments to distinguish code and skill hypotheses, and PhysicalRSI selects harness successors through embodied evaluation [27–29]. We separate experience acquisition from execution, testing whether knowledge from one scene helps the same or a different model solve new layouts of the same task.

3 EmbodiedEvo

3.1 Code as World

Code as World organizes interaction with the physical environment into two levels of code. L1 provides the basic operations for observing and acting; L2 combines these operations into executable Python procedures. Figure 2 places this programmatic control within the broader set of embodied control paradigms.

L1: environment primitives. The environment exposes four primitives for reading the task, observing the scene, executing an action, and resetting the environment (Table 1). These operations form the fixed interface available to the agent. Reset is restricted to Playground, where it enables another attempt on the same scene.

In RoboDojo, observations contain RGB images and robot state, including joint states, end-effector poses, and gripper command states. Actions specify joint or end-effector targets together with gripper commands; the native controller handles inverse kinematics and interpolation. Object ground-truth poses, depth, and camera calibration are unavailable through this interface.

L2: programs composed from primitives. The agent writes Python programs that combine L1 calls through variables, functions, loops, and conditional branches. For example, a grasping procedure can read

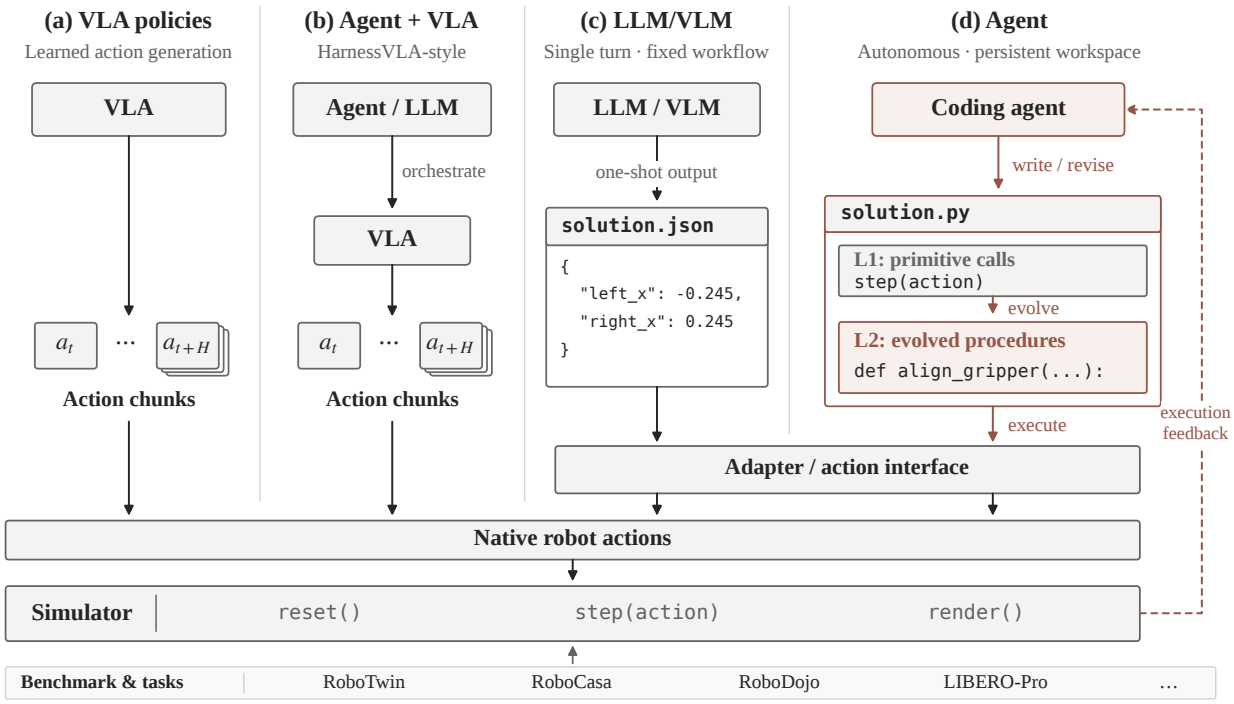


Figure 2 Comparison of embodied control paradigms. (a) A VLA policy emits native action chunks. (b) An agent orchestrates a VLA policy while retaining its action-generation role. (c) A single-turn LLM/VLM produces a structured action specification for an adapter. (d) A coding agent develops and revises executable programs through environmental feedback, composing primitive calls (L1) into evolved procedures (L2). The dashed loop shows execution feedback for program revision.

Table 1 Native environment interface. The first three primitives are available in both phases; reset is restricted to Playground phase.

Primitive	Function
`get_instruction()`	Returns the current task and goal.
`get_observation()`	Returns camera images and robot state without advancing physics.
`step(action)`	Executes a native action and returns the resulting observation, completion signal, and termination status.
`reset()`	Restores the same scene for another exploration attempt and renews its native action allowance.

the current robot state, move towards a target, close the gripper, and lift. It can accept target coordinates as arguments and check the observed end-effector pose before continuing. A single program can therefore coordinate multiple native actions, while its functions can be reused in later programs.

This separation fixes what the robot can observe and execute while letting the agent decide how to compose those capabilities. Code makes the sequence, parameters, and checks explicit, so the agent can inspect and modify the procedure as the task progresses. Figure 3 shows how these programs connect to a task environment. The program-level entry point `step(code)` executes submitted Python through an adapter, and the program issues native `step(action)` calls to the selected simulator. The adapter keeps the interface consistent across task backends; our experiments use RoboDojo.

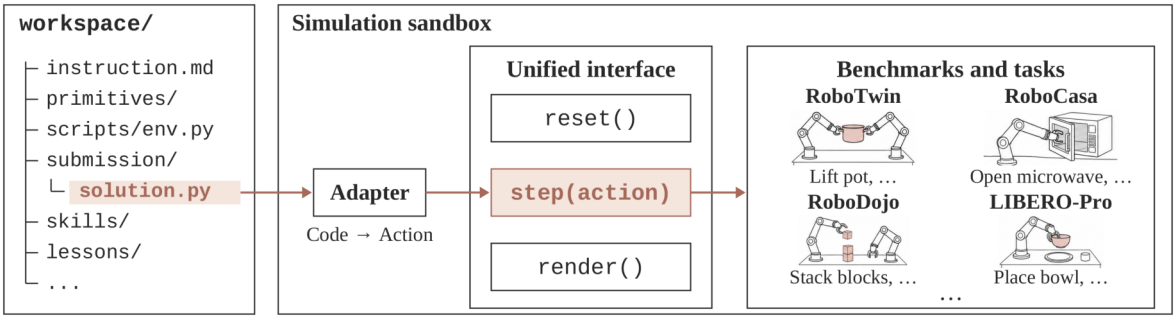


Figure 3 Code-to-task mapping through a unified simulation interface. The agent develops `submission/solution.py` within its workspace and submits it to an adapter. The adapter executes the program and issues native `step(action)` calls through the unified interface to the selected benchmark backend.

3.2 Agent as Policy

Agent as Policy assigns the control decisions to the coding agent: what to observe, which program to execute, and how to revise it after feedback. Using the L1/L2 interface, the agent directs an iterative process of inspection, programming, execution, and repair (Figure 2d). It chooses the task decomposition and the timing of each step from the current observations and execution history.

A workspace for acting through code. The agent works in an isolated filesystem workspace (Table 2). Task instructions and interface documentation provide the fixed specification; source files and scratch space support program development. Execution records preserve what ran and what happened, allowing the agent to inspect earlier attempts. Skills and Lessons provide the persistent knowledge introduced in Section 3.4.

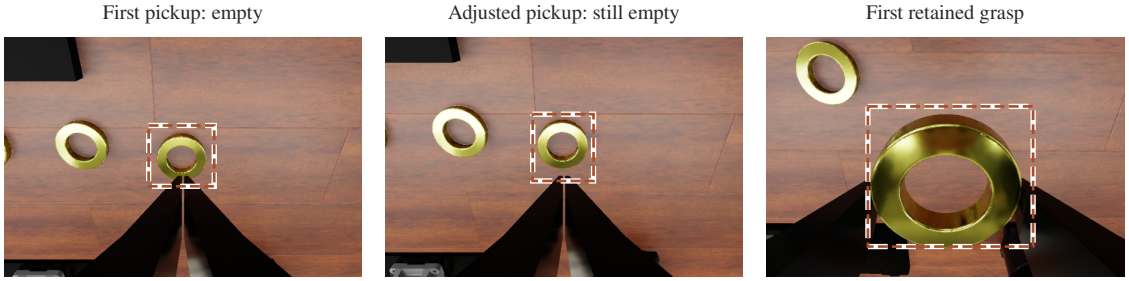
Table 2 Software-engineering workspace. Paths are relative to the agent's workspace root. Protocol files and execution records are read-only to the agent.

Component	Role and access
`instruction.md`	Fixed task delegation, phase rules, and budgets.
`primitives/primitives.md`	Fixed documentation of the robot interface.
`submission/solution.py`	Editable Python source for the next control segment.
`scripts/env.py`	Fixed client for execution, status queries, and session completion.
`observations/`	Host-written history of submitted source, observations, video, and execution logs.
`scratch/`	Writable plans, calculations, and temporary analyses.
`skills/`, `lessons/`	Reusable procedures and diagnostic knowledge; writable in Playground and read-only in Test.

An agent-directed interaction loop. The agent reads the task and inspects the scene, edits `submission/solution.py`, and submits it through the execution client (Figure 3). After execution, it can examine the submitted source, camera observations, standard output and errors, completion signal, and remaining budgets. It then decides whether to gather another observation, adjust a target, revise a function, continue to the next stage, or finish the episode. For example, after observing a dropped object, it can inspect the new scene and revise the grasp before attempting another lift. These choices are made within one coding-agent session.

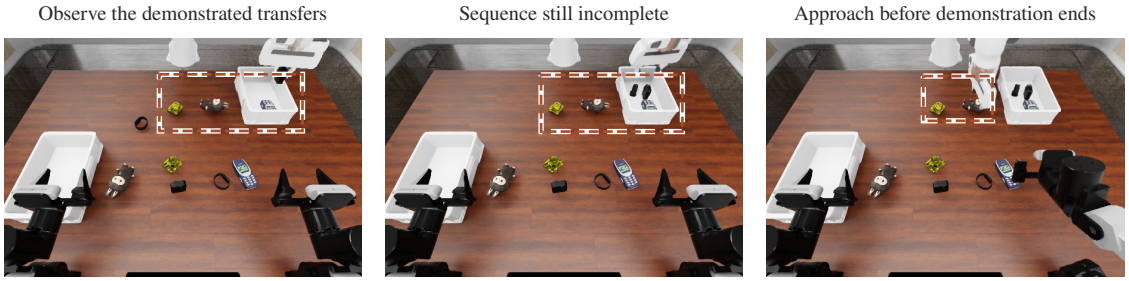
Adapting to the current physical state. Robot state, Python variables, and function definitions persist between submissions. Editing code or encountering an error does not undo completed actions, so the agent must adapt its next program to the resulting scene. Playground permits same-scene resets and renews the native action allowance, while execution slots and elapsed time remain cumulative. Test disables reset, requiring recovery within the current scene and remaining budget. The workspace thus supports local program revision throughout an episode; Section 3.4 explains how experience can also be retained across episodes.

(a) Tic-tac-toe: repeated empty lifts consume the action budget



The arm reaches its targets, but useful contact must still be established.

(b) Imitation sorting: acting before the demonstration ends



Knowing the next object does not establish that manipulation can start.

Task	Score	SR (%)
Tic-tac-toe	55	0
Imitation sorting	10	0

Figure 4 Two failure trajectories of Astra. (a) Two empty lifts precede the first retained grasp; the episode later exhausts its budget without completion. (b) Observation switches to manipulation during an unfinished demonstration, followed by termination. Boxes identify relevant objects. The table reports Astra's averages over all tested scenes of each task.

3.3 Deep Dive into Astra's Failures

To understand where coding agents struggle in physical interaction, we evaluate GPT-6-Astra with xhigh reasoning effort in the code workspace described in Section 3.2. The evaluation is on RoboDojo-sim tasks spanning Generalization, Precision, Long-Horizon, Memory, and Open, with 5 scenes per task. Astra receives the task description and robot-interface documentation, without the task wiki described in RoboProbe [30]. Each episode starts with a fresh agent, workspace, and simulator, permits up to 100 accepted code executions within the task's native action allowance, and disables reset.

Astra achieves an average success rate of 8.50%. We examine its submitted programs, camera observations, and feedback to locate where plausible plans fail during physical execution. Figure 4 shows two illustrative episodes, each exposing a different gap between issuing a valid command and advancing the task.

A correct action sequence still lacks a reliable manipulation routine. In tic-tac-toe, Astra attempts to pick up a ring and place it on the board. The gripper reaches its target positions, yet two successive attempts leave the ring on the table (actions 216 and 283). After inspecting a wrist-camera view and adjusting the grasp, Astra first visibly retains the ring at action 367, consuming one-third of its 1,100-action budget. The episode later times out without completing the board. This trace shows how constructing a local manipulation routine can compete with task completion. It motivates retaining a parameterized procedure that combines approach, alignment, closure, and verification of object pickup, so that subsequent attempts can build on a working operation.

Action decisions lack explicit precondition checks. In imitation sorting, Astra has identified only the first three objects at action 240, yet starts approaching the first *while the demonstration continues*. The episode terminates 16 actions later, leaving 1,344 actions unused. The visible gap is acting without checking whether the demonstration has finished. Identifying a target alone does not establish readiness to manipulate it. This motivates retaining observation-grounded guidance on what to verify before acting, with unresolved explanations marked as hypotheses. Here, the trace supports checking demonstration completion; it does not prove that early movement caused termination or that waiting would ensure success.

The two failures expose complementary needs: (1) reliable procedures for carrying out an operation, and (2) explicit conditions for deciding when to attempt it. Within an episode, physical feedback can reveal both, but each fresh Astra agent starts again from the interface and task description. This motivates a mechanism that turns local corrections into reusable procedures and retains failure conditions to guide later decisions. Section 3.4 develops that mechanism through harness evolution.

3.4 Self-Improvement through Harness Evolution

EmbodiedEvo addresses the two needs in Section 3.3 by retaining *Skills* and *Lessons* in an evolving harness (Figure 5). Skills combine fixed native primitives into reusable control procedures, such as grasping with a checked lift. Lessons record applicability conditions, observed failure modes, and guidance for diagnosis and recovery. Together, they capture how to carry out an operation and what to check before or after it. Self-improvement updates these artifacts while model parameters, task goals, and the robot interface remain fixed.

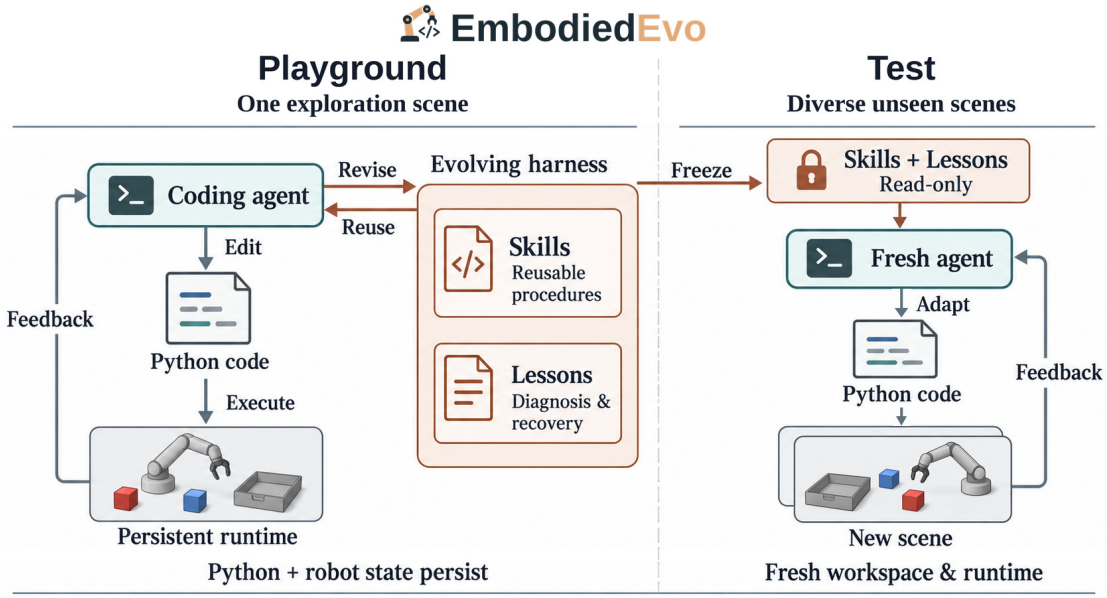


Figure 5 EmbodiedEvo framework. Left: on one Playground scene, the agent edits and executes code, uses feedback to revise Skills and Lessons, and reuses them in subsequent attempts. The Freeze arrow transfers these artifacts to Test. Right: each unseen scene starts with a fresh agent and runtime; code adapts through feedback while the harness stays read-only. Robot scenes are schematic.

On the left of Figure 5, one Playground scene hosts a persistent loop of code editing, execution, and feedback. Robot state and Python definitions persist between submissions; reset allows another attempt on the same scene. The Revise and Reuse arrows show how the agent updates Skills and Lessons and applies them in subsequent attempts. A successful operation can contribute a parameterized procedure; an unsuccessful attempt can reveal a useful condition or suggest a recovery strategy. Even an unfinished task can yield useful findings; tentative explanations remain hypotheses.

The Freeze arrow marks the end of Playground: only Skills and Lessons are carried into Test as read-only artifacts. On the right, each unseen scene of the same task starts with a fresh agent, workspace, Python

namespace, and simulator. The agent uses the frozen harness to construct code and adapts that code through execution feedback. Reset is disabled, and Test episodes do not update the harness or share new experience. This separates within-scene adaptation from the contribution of knowledge acquired during Playground.

The same frozen artifacts can be supplied to another executor because their contents are explicit procedures and guidance. Cross-model reuse requires no additional Playground exploration, parameter updates, or learned adapter. We compare self-generated and transferred harnesses under the same executor and Test conditions to assess whether the acquired knowledge remains useful across models.

4 Experiments

We evaluate EmbodiedEvo on RoboDojo-Sim to address the following research questions: **(1)** How well does EmbodiedEvo perform compared with VLA- and WAM-based methods? **(2)** Which capabilities improve through harness evolution, and which tasks remain difficult? **(3)** Can acquired experience transfer across models and tasks?

4.1 Experimental Setup

Tasks and agents. RoboDojo-Sim [31] is a challenging benchmark for evaluating generalist robot manipulation policies. It comprises 12 Generalization, 8 Precision, 8 Long-Horizon, 6 Memory, and 8 Open tasks. Each harness(skills + lessons) is acquired on one Playground scene layout distinct from the Test layouts. Executors are GPT-6-Astra at xhigh and medium effort, GPT-5.6-sol at xhigh, and GPT-5.6-luna at xhigh, abbreviated Astra, Sol, and Luna.

Evaluation scales. We use two evaluation scales: (1) dev set: 5 scenes per task (210 episodes per condition) for input comparisons, ablations, and cross-model analyses; and (2) full set: 50 scenes per task (2,100 episodes, one seed) for Full Evaluation of the complete benchmark. Paired comparisons use identical scenes. For each Generalization task, the two scales use three standard and two random scenes, and 25 standard and 25 random scenes, respectively. Scene identities follow the public evaluation records [32]. Cross-task transfer uses a fixed subset of 12 target tasks and is reported separately from the full-benchmark averages.

Inputs and budgets. The task description states the goal in one sentence; the official wiki provides additional task rules and operational guidance [33]. Full Evaluation uses *task description + wiki* to align task information with the official GPT-6-Astra evaluation [7]. The input ablation compares both settings; all other paired analyses use *pure task description*. Each Playground chain and Test episode allows up to 100 accepted code executions and retains the task's native action budget. Playground permits same-scene resets; acquired Skills and Lessons are then frozen. Test starts a fresh agent, workspace, and simulator per scene, with read-only experience and no reset.

Metrics. We report Score (0–100) and success rate (SR, %). Score quantifies task completion, including partial progress, according to the benchmark's task-specific scoring rules. SR is the percentage of episodes that satisfy the task's success criterion.

4.2 Overall Performance

We evaluate EmbodiedEvo with Astra/xhigh on the full RoboDojo-Sim set. We compare category-balanced Score and SR with the VLA, WAM, and agent-based results in the October 9, 2026 leaderboard snapshot.

Table 3 shows that EmbodiedEvo achieves the highest Average Score (44.48) and SR (37.82%) among the listed methods. It exceeds VPP2, the strongest listed WAM in aggregate, by 5.22 Score points and 5.56 SR percentage points. Against Simate-beta (VLA), the margins are 10.53 and 9.86 points, respectively; it also exceeds PhysicalRSI (Agent + VLA) by 8.21 and 6.44 points.

Table 3 Full RoboDojo simulation evaluation. EmbodiedEvo alongside selected systems from the October 9, 2026 leaderboard snapshot, ordered by increasing Average Score. Score and success rate (SR, %) are higher-is-better; bold marks the best value in each metric column.

Model	Type	Average		Generalization		Precision		Long-horizon		Memory		Open	
		Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
GPT-6-Astra	Agent	28.97	22.48	33.36	**30.50**	12.65	4.00	21.45	8.25	43.04	38.67	34.36	31.00
InternW0-Δ	WAM	30.77	23.91	30.09	22.78	31.98	23.25	46.29	29.33	34.67	34.00	10.84	10.17
Simate-beta	VLA	33.95	27.96	35.09	27.95	34.35	26.92	**57.84**	**43.42**	33.33	33.00	9.12	8.50
Awomo-0.5	WAM	35.34	29.64	30.31	23.84	39.65	33.50	40.36	26.75	42.27	41.22	24.09	22.92
PhysicalRSI	Agent + VLA	36.27	31.38	21.30	15.62	38.05	32.50	46.37	37.33	46.74	46.56	28.88	24.92
VPP2	WAM	39.26	32.26	**36.38**	29.50	**43.03**	**35.17**	56.28	37.00	56.01	55.56	4.61	4.08
EmbodiedEvo (Ours)	Agent	**44.48**	**37.82**	34.12	29.67	23.65	13.25	37.20	23.50	**71.90**	**69.67**	**55.52**	**53.00**

EmbodiedEvo leads on Memory and Open tasks by large margins. EmbodiedEvo achieves 71.90 Score / 69.67% SR on Memory and 55.52 / 53.00% on Open. Its SR exceeds the next-best listed method in each category: VPP2 at 55.56% on Memory and GPT-6-Astra at 31.00% on Open. The cases in Section 4.3 suggest why Skills and Lessons suit these tasks. Skills provide reusable grasping, carrying, and placement procedures that can be composed around the current instruction; Lessons guide observation, sequence tracking, and checks before advancing to the next phase. Together, they connect remembering what the current scene requires with procedures for executing it, supporting memory-dependent sequences and instruction-conditioned manipulation.

Agent still struggles with precise manipulation and long-horizon execution. Its Precision SR is 13.25%, below VPP2's 35.17%, while its Long-Horizon SR is 23.50%, below Simate-beta's 43.42%. These tasks require accurate contact and reliable execution across successive operations. The failure cases in Section 4.3 show that reaching a commanded pose can leave a grasp unsecured, and that completing some stages can leave the overall task unfinished. Retained procedures and guidance therefore remain dependent on reliable physical execution: contact accuracy and maintaining successful manipulation across a sequence are continuing bottlenecks.

4.3 What Does Harness Evolution Actually Change?

Research questions

(1) What can a coding agent already solve without evolution? **(2)** What new capabilities does evolution add? **(3)** What remains unsolved after evolution?

To answer the questions above, we conduct paired evaluations of Astra/xhigh with and without evolved experience, using identical scenes and task inputs. We test both wiki settings [33], then examine task outcomes and execution traces to distinguish existing competence, newly acquired capabilities, and remaining failures. Due to the cost of large-scale agent evaluation, we conduct this and all subsequent experiments on the *dev set (5 scenes per task)*.

EmbodiedEvo consistently improves performance across different input settings. In Table 4, EmbodiedEvo improves Average Score by 23.87 points and SR by 21.67 percentage points without wiki. With wiki supplied to both methods, the gains remain 21.75 Score points and 21.72 SR points. Figure 6 shows positive Score and SR gains across all five categories under both inputs. These gains suggest that experience contributes execution knowledge under either input setting; the following traces show how Skills supply tested operations and Lessons guide their use.

Table 4 Task knowledge and inherited experience. All rows use Astra/xhigh on matched scenes. Each cell reports Score / SR (%).

Method	Input	Average		Generalization		Memory		Precision		Long-horizon		Open	
		Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
Astra	Task description	14.77	8.50	3.19	0.00	12.67	10.00	11.38	5.00	18.12	0.00	28.50	27.50
EmbodiedEvo (Astra)	Task description	**38.64**	**30.17**	22.36	16.67	59.33	56.67	29.88	15.00	29.50	12.50	52.12	50.00
Astra	Task description + wiki	20.67	13.36	12.33	7.64	39.67	36.67	15.00	7.50	19.38	0.00	17.00	15.00
EmbodiedEvo (Astra)	Task description + wiki	**42.42**	**35.08**	36.15	31.25	70.33	66.67	23.12	12.50	33.75	20.00	48.75	45.00

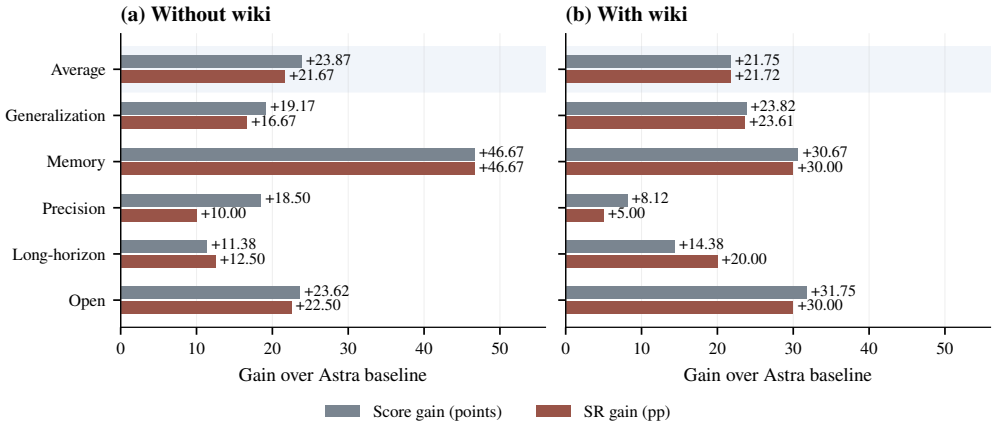


Figure 6 Effect of EmbodiedEvo with and without wiki. Score (points) and SR (percentage points) gains over Astra/xhigh on matched scenes and inputs: (a) task description only; (b) task description + wiki.

Table 5 Task performance before and after harness evolution. Selected tasks with the same Astra/xhigh executor and matched dev scenes. Bold marks the higher value within each before/after pair; ΔSR is the change in percentage points. Groups describe observed success in these scenes.

Task	Category	Astra		EmbodiedEvo (Astra)		ΔSR (pp)
		Score	SR (%)	Score	SR (%)	
(1) Successful before evolution						
Align blocks	Open	60	60	**80**	**80**	+20
Conveyor matching	Memory	60	60	**100**	**100**	+40
(2) Successful only after evolution						
Stack blocks	Generalization	10	0	**100**	**100**	+100
Tic-tac-toe	Long-Horizon	55	0	**100**	**100**	+100
Imitate sorting	Memory	10	0	**90**	**80**	+80
(3) Unsuccessful in both conditions						
Insert tubes	Precision	4	0	**36**	0	0
Insert key	Precision	9	0	**15**	0	0

We next examine the operations and decisions behind these gains. Table 5 organizes representative tasks into existing competence, newly acquired capability, and remaining limits. Appendix B provides the complete dev- and full-set results.

Visible goals and locally correctable actions can already be solved without evolution. Astra achieves 60% SR on both block alignment and conveyor matching; evolution raises these to 80% and 100%, respectively. Both tasks provide visible targets and feedback for adjusting the next action, allowing online coding and

observation to support local corrections. Here, evolution improves the reliability of operations Astra can already perform.

Evolution supplies reusable operations and conditions for advancing through a task. Tic-tac-toe and block stacking rise from zero to 100% SR; imitation sorting rises from zero to 80%. These tasks require more than identifying a spatial goal: successful local operations must be repeated, or action must wait until the necessary scene information has been collected.

Figure 7 contrasts two such decision points on matched scenes. In tic-tac-toe, the baseline repeatedly lifts without retaining a ring, eventually scoring 50 with no success. EmbodiedEvo executes the inherited `rim_pick` routine, retains a ring at action 38, and completes the board with Score 100. The useful change is an executable grasp procedure that supports repeated placements. In imitation sorting, the baseline starts moving before the demonstration finishes and terminates with Score 0. The evolved agent follows its observation lesson, records the current sequence through action 455, and reproduces it successfully. The order differs from Playground, so the reusable knowledge concerns *when to act*, rather than a memorized object sequence. Together, these cases connect Skills to repeatable physical operations and Lessons to the conditions for using them. They illustrate the observed mechanisms; the paired runs do not isolate one artifact's causal effect. Appendix D.1 provides the supporting records, and Appendix F gives a complete successful language-stacking trace.

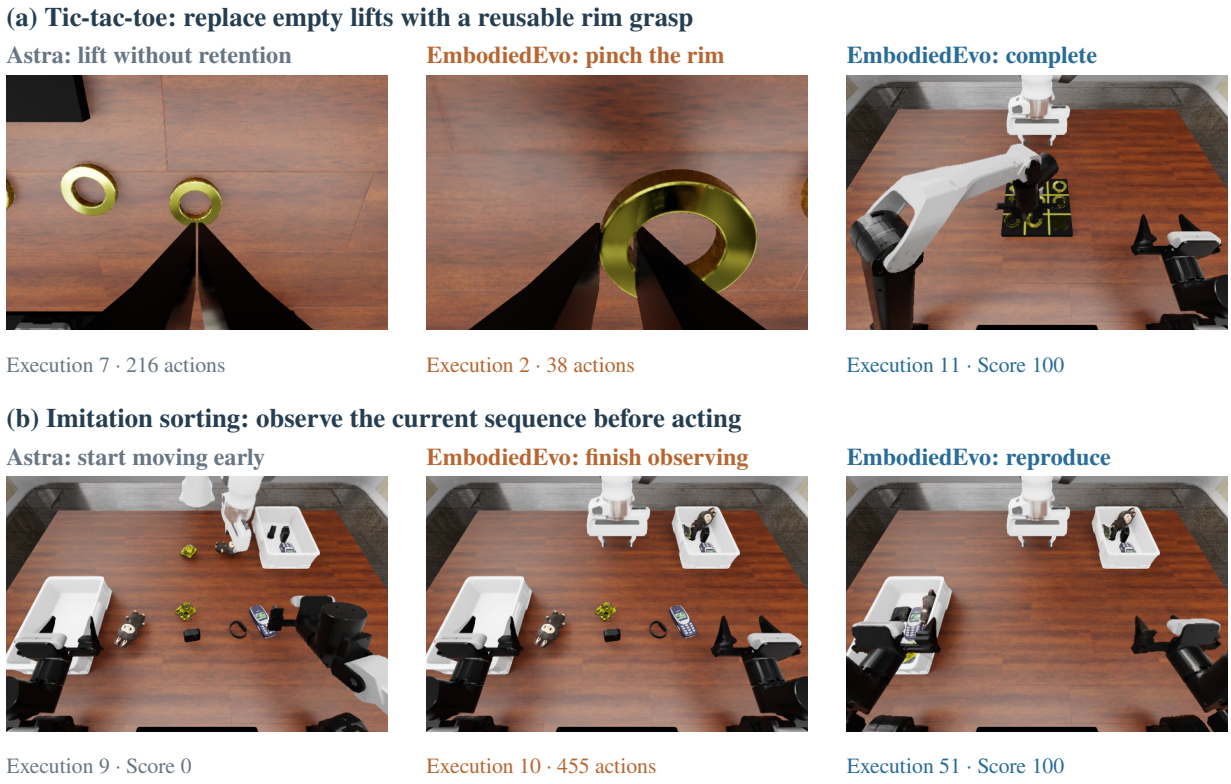


Figure 7 New capability: changes at the decisive operation or phase. Each row compares Astra and EmbodiedEvo on the same task and scene. (a) Tic-tac-toe, layout 27: empty pickup versus an inherited rim grasp and completed board. The first pickups use different rings and arms. (b) Imitation sorting, layout 4: early motion versus observing the complete new order before executing it. Both timing and motion differ; the baseline feedback does not identify the termination cause. Frames show selected decision points, not synchronized times.

Reusable motion does not guarantee precise contact. Insert tubes improves from 4 to 36 Score and insert key from 9 to 15, yet neither achieves a dev-set success. Figure 8 localizes the key failure: bounded motion retains the key during pickup, but the receiving hand closes empty during handover. Subsequent corrections do not secure it; releasing the donor drops the key at the action limit, *before insertion*. A working pickup procedure therefore leaves the next contact transition unresolved. This gap between successful subroutines and complete manipulation explains why partial Score gains need not become SR gains on Precision tasks.



Figure 8 Remaining limit: unsuccessful contact transfer. EmbodiedEvo, insert key, layout 44. The donor retains the key, but the receiver closes without grasping the shaft (highlighted); the final release loses the key (highlighted). Wrist observations and gripper feedback locate the failure at handover, before insertion. This episode scores 15 with no success, matching the task's dev-set mean.

4.4 Transferring Evolved Experience across Models and Tasks

Research questions

(1) Can experience acquired by a stronger model improve weaker executors? **(2)** Can experience from other tasks improve performance on the current task?

To answer the questions above, we conduct matched comparisons of No Experience, self-evolved harnesses, and Astra/xhigh harnesses for each receiving executor. We also fix Astra/xhigh on twelve target tasks and compare same-task, related-task, and mismatched-task harnesses against No Experience. All transferred Skills and Lessons are frozen, with no further exploration by the receiver.

Stronger experience acquisition benefits weaker executors. Table 6 and Figure 9 compare No Experience (No Evolution), Self-Evolved Harness, and Harness from Astra/xhigh at a fixed executor. Relative to No Experience, Astra/xhigh experience raises SR by 18.72 points for Astra/medium, 8.86 for Sol/xhigh, and 5.50 for Luna/xhigh. Sol reaches 12.03% SR versus 2.50% with its own harness; Luna reaches 5.50% versus zero. Astra/medium tests a change in source reasoning effort, while Sol and Luna test a change in source model. These gains separate the ability to acquire useful experience from the ability to apply it: a weaker executor can use procedures and conditions that its own exploration failed to establish.

Transferred experience supplies actionable conditions for control. On `cover_blocks`, Sol fails with its own harness after 680 native actions, but succeeds with Astra/xhigh's harness in 678 actions (Figure 13b). Its own procedure loses a cup during lateral motion; the transferred harness distinguishes the grasp height for an empty cup from that for a cup covering a block. Sol applies these conditions and adapts the color order to the current scene. The case links successful reuse to selecting an appropriate operation and its place in the sequence, with nearly unchanged action expenditure.

Other-task experience improves performance without target-task exploration. On twelve fixed targets, related-task experience adds 14.18 Score points and 14.72 SR percentage points over No Experience; mismatched-task experience adds 18.65 and 16.67, respectively (Figure 10). Same-task experience provides a reference gain of 24.43 Score points and 21.39 SR points. Both transfer settings reuse existing artifacts

Table 6 Performance under three experience conditions. Score and success rate (SR, %) are higher-is-better. Each executor uses matched scenes across conditions. Self-evolved harnesses are acquired by the listed executor; the final group reuses frozen experience from Astra/xhigh.

Executor	Average		Generalization		Precision		Long-horizon		Memory		Open	
	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
No Experience (No Evolution)												
Astra / xhigh	14.77	8.50	3.19	0.00	11.38	5.00	18.12	0.00	12.67	10.00	28.50	27.50
Astra / medium	6.58	4.33	1.81	0.00	2.88	0.00	5.88	0.00	7.33	6.67	15.00	15.00
Sol / xhigh	4.10	3.17	0.56	0.00	3.00	2.50	3.12	0.00	3.33	3.33	10.50	10.00
Luna / xhigh	0.08	0.00	0.42	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Self-Evolved Harness												
Astra / xhigh	**38.64**	**30.17**	**22.36**	**16.67**	**29.88**	**15.00**	**29.50**	**12.50**	**59.33**	**56.67**	**52.12**	**50.00**
Astra / medium	16.05	8.67	6.49	0.00	6.38	0.00	16.50	0.00	28.00	23.33	22.88	20.00
Sol / xhigh	4.09	2.50	0.28	0.00	0.38	0.00	12.12	5.00	0.17	0.00	7.50	7.50
Luna / xhigh	0.62	0.00	0.00	0.00	0.00	0.00	3.12	0.00	0.00	0.00	0.00	0.00
Harness from Astra/xhigh												
Astra / medium	30.60	23.06	17.67	15.28	11.38	0.00	26.12	12.50	55.83	50.00	42.00	37.50
Sol / xhigh	14.09	12.03	4.51	3.47	2.75	0.00	15.38	10.00	27.33	26.67	20.50	20.00
Luna / xhigh	6.55	5.50	0.21	0.00	0.38	0.00	4.50	0.00	20.17	20.00	7.50	7.50

Notes: Executor labels include the reasoning effort. Generalization variants, tasks within each category, and the five categories are equally weighted at their respective levels. Scene selection and evaluation details are summarized in Appendix B.

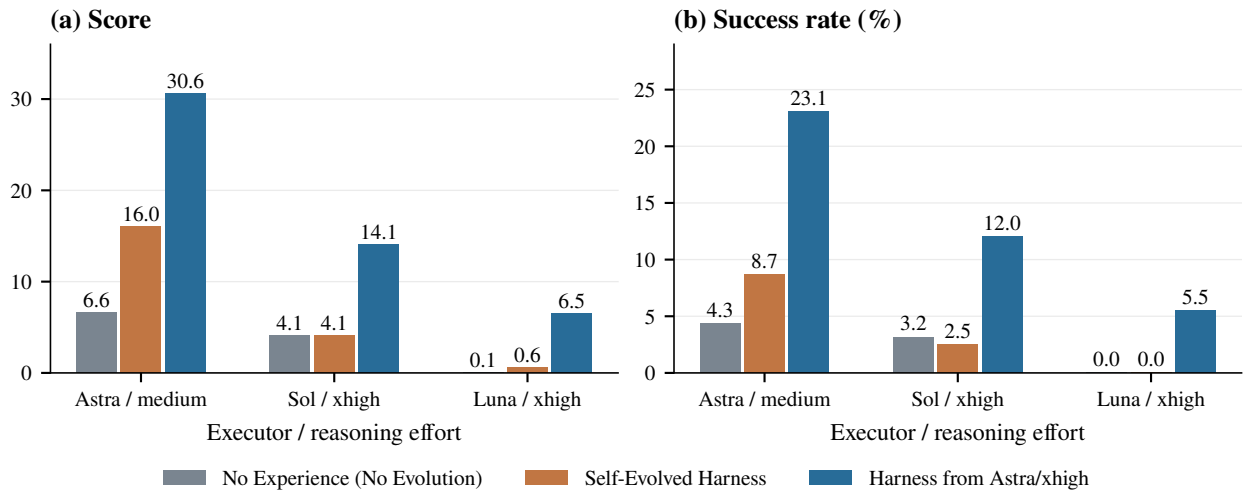


Figure 9 Experience transfer to weaker executors. Each executor is evaluated on the same scenes under the three conditions used in Table 6: No Experience (No Evolution), Self-Evolved Harness, and Harness from Astra/xhigh. The latter reuses frozen experience without additional exploration. Astra/medium compares source reasoning strengths within the same model; Sol/xhigh and Luna/xhigh test cross-model reuse.

without exploring the target. Their gains are consistent with reuse of general control knowledge, including bounded motion, retention checks, and terminal return-home checks, that can guide execution across different tasks.

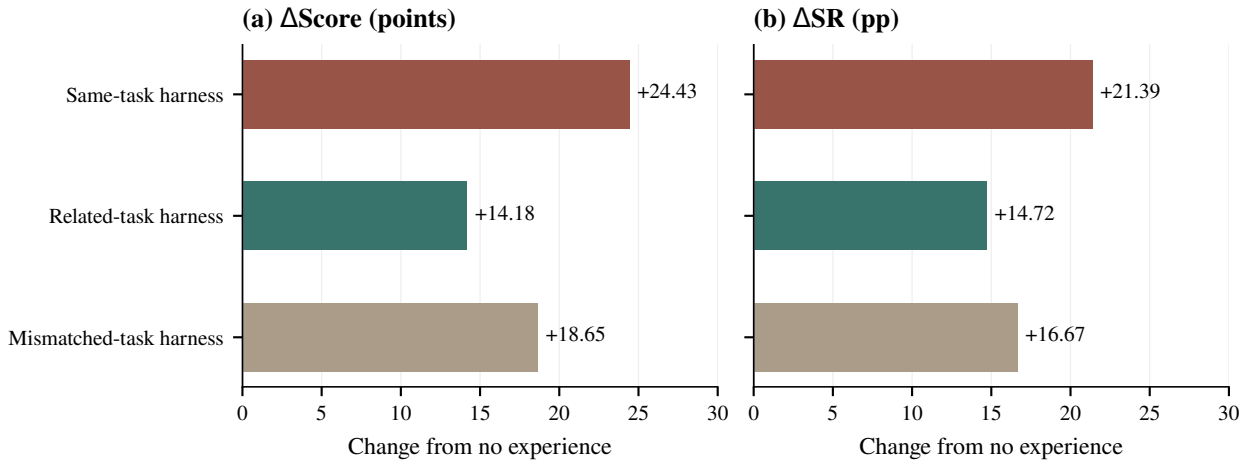


Figure 10 Performance gains from experience across tasks. Each bar is the change from No Experience on the same target scenes, with Astra/xhigh fixed. ΔScore uses points and ΔSR uses percentage points; zero denotes the No Experience reference. Gains weight the twelve targets equally after combining Generalization variants. Related-task and mismatched-task harnesses require no new target-task exploration; the same-task harness is shown as a reference.

Task-label similarity alone does not predict transfer benefit. Mismatched transfer exceeds related transfer by 4.47 Score points and 1.94 SR points in this target set. The direction varies by task: on `swap_T`, mismatched and related sources yield ΔSR of +100 and 0 points; on `insert_tubes`, they yield 0 and +40 points. The two mappings reuse the same source pool with equal aggregate file volume (Appendix C). These reversals suggest that selecting useful experience requires inspecting the retained operations and their applicability conditions, beyond the semantic similarity of task labels.

4.5 How Experience Representation and Content Shape Reuse

Research questions

(1) Can compact experience preserve the benefits of full interaction history? **(2)** Can an agent acquire useful knowledge without completing the task? **(3)** What makes retained knowledge useful for later execution?

To answer the questions above, we conduct a controlled comparison of Skills/Lessons and Raw History from the same Playground, using Astra/xhigh on matched Test scenes. We measure task performance and total token use, then inspect artifacts from unfinished tasks and link saved procedures and guidance to subsequent executions.

Skills/Lessons achieve comparable task performance with fewer tokens. Raw History provides the ordered programs and public feedback from the same Playground. It reaches 36.41 Score / 29.78% SR, compared with 38.64 / 30.17% for Skills/Lessons (Figure 11). Mean total Test usage falls from 4.460M to 3.958M tokens per episode, a reduction of 11.3%. Input usage drops from 4.445M to 3.939M, including an 87.9% reduction in uncached input. Skills/Lessons organize exploration into reusable procedures and explicit guidance that the receiver can consult directly. The savings arise primarily on the input side, showing how this representation makes prior interaction more economical to use. Appendix E gives the paired estimates and token accounting.

Unfinished Playground tasks can still yield useful experience. On `pick_from_conveyor_by_image`, Astra records no Playground success across 100 executions, yet its frozen harness raises Test Score from 0 to 40 and SR from 0 to 40% (Table 7). The exploration establishes useful local operations: execution

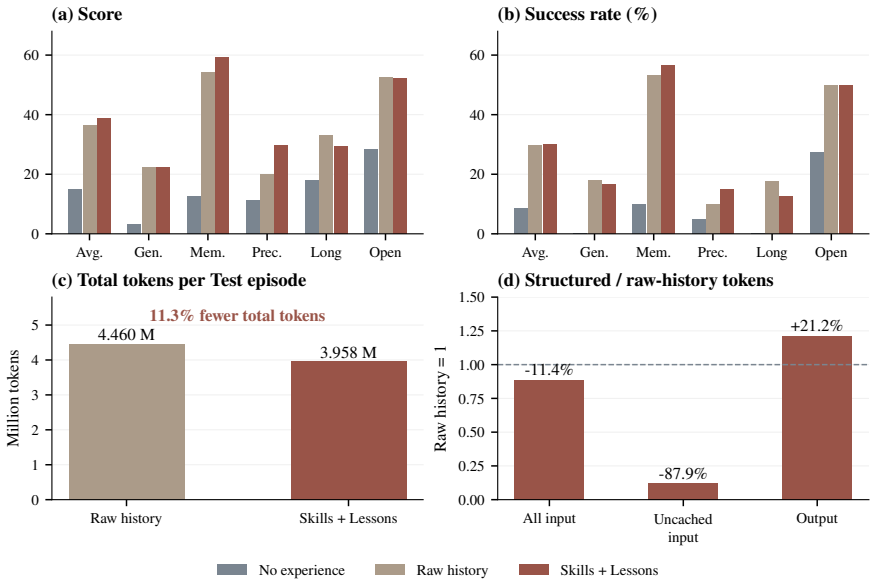


Figure 11 Comparable performance with lower token use. The executor, source Playground, and Test scenes are fixed. (a–b) Raw History and Skills/Lessons achieve similar observed Score and SR. (c) Skills/Lessons reduce total Test tokens by 11.3%. (d) The reduction comes from input usage, while output increases. Tokens are cumulative per episode, with cache reads included in input.

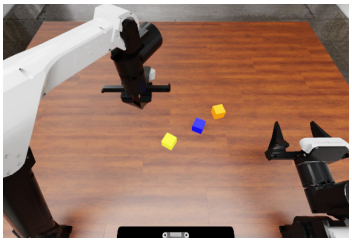
Table 7 Useful experience without Playground task success. Astra/xhigh on image-conditioned conveyor pickup; frozen experience is evaluated on new scenes.

Exploration	**Observed progress and retained knowledge**
Task outcome	No task success across 100 code executions.
Local progress	Execution 74: tracked grasp and 6 cm lift in 31 native actions.
Skills	Track a moving target through closure; monitor contact during transport.
Lessons	Anticipate conveyor motion; verify object retention after a short lift.
Test condition	**Score / SR (%)**
No experience	0 / 0
Skills/Lessons	**40 / 40**

74 completes a tracked grasp and test lift in 31 native actions. The retained files provide moving-target capture, contact-guarded transport, and guidance to update targets as the conveyor moves. These findings can be carried forward even when the entire task remains unfinished. Experience accumulates at the level of operations and their operating conditions, allowing later agents to build on partial progress.

Useful experience couples executable corrections with conditions and checks. In the stacking trace, Test execution 9 reuses a controller with an 8 mm translation bound; the agent checks the retained load and completes the stack at execution 15 (Figure 12). In covering, Astra validates a higher uncover grasp on three cups and records separate heights for empty-cup pickup and uncovering; Sol applies these conditions and succeeds at execution 17. These examples connect a concrete correction, a condition for applying it, and an observable check of its effect. Skills supply the operation, while Lessons guide its selection and verification in the current scene. Appendix D.2 provides the retained excerpts and reuse records.

(a) Playground feedback



Execution 16
Block lost during carry

(b) Frozen lesson + skill

"The arm reached its target,
but load retention failed."

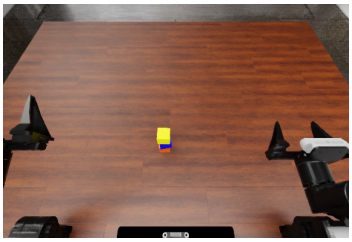
Abridged controller

```
if distance > max_step:
    commanded = (
        current + delta
        * max_step / distance
    )
```

Test call · execution 9

```
max_translation_step=0.008
```

(c) Test execution



Execution 15
Stack completed (Score 100)

Figure 12 A correction retained with an execution check. Left: Playground execution 16 records a dropped load. Center: the frozen lesson distinguishes reached pose from retained load, and the controller limits motion using the observed pose. Test execution 9 calls it with an 8 mm bound. Right: the later successful stack at execution 15. The controller is abridged pseudocode; the lesson excerpt and call parameter come from the retained artifacts and Test program.

4.6 Performance Gains and the Cost of Reusing Experience

Research questions

(1) What execution cost accompanies the gains from experience? **(2)** Can structuring experience lower the cost of reuse?

To answer the questions above, we compare No Experience, Skills/Lessons, and Raw History with Astra/xhigh on matched scenes. Skills/Lessons and Raw History share the same Playground source. We report per-episode model expense and interaction counts, including failures, using a fixed token-rate schedule and excluding Playground acquisition.

Experience improves performance with higher execution expense. Score rises from 14.77 to 38.64 and SR from 8.50% to 30.17%, while mean total Test cost increases from $4.058 to $5.732 per episode (+41.3%; Table 8). The executor consults retained procedures and guidance while continuing to observe, submit code, and check feedback. Experience therefore improves completion while still requiring sustained interaction with the environment.

Table 8 Performance and execution expense at a fixed executor. Astra/xhigh on matched dev scenes. Costs and interaction counts are per-episode means, including failures. Raw History and Skills/Lessons use the same Playground source.

Experience	Score	SR (%)	Cost (USD)	Code executions	Native actions
No Experience	14.77	8.50	4.058	22.3	723.0
Raw History	36.41	29.78	12.203	23.7	705.1
Skills/Lessons	38.64	30.17	5.732	29.0	722.6

Structured experience is cheaper to reuse than raw history. Using the same Playground source, Skills/Lessons cost $5.732 per Test episode versus $12.203 for Raw History, a 53.0% reduction, with SRs of 30.17% and 29.78%, respectively. Skills/Lessons retain reusable procedures and conditions for applying them, reducing the input processed when consulting prior exploration. This compact representation preserves similar observed performance at a lower total execution cost.

5 Discussion

Code makes physical knowledge inspectable and revisable. Code exposes the parameters, checks, and control flow through which a physical strategy is executed. This gives feedback a concrete target for revision: after a dropped load, the stacking agent retains bounded motion and checks object retention during later execution. Parameterized procedures can then preserve a useful operation while allowing its targets to change with the scene. Lessons complement this representation by recording when to apply or reconsider it. Raw History achieves similar observed success, while Skills/Lessons reduce total Test token use by 11.3% . This suggests that organizing interaction into accessible procedures and guidance is a practical benefit of the representation; code alone does not guarantee valid physical assumptions.

Evolution changes the starting point of task solving. With the model and robot interface fixed, evolution changes which operations and decision conditions are available before a new episode begins. The tic-tac-toe and sorting cases illustrate two consequences: a reusable grasp reduces repeated local search, while retained guidance helps the agent decide when to move from observation to manipulation. These cases suggest that evolution shifts part of the work from discovering an operation to adapting it to the current scene. The remaining zero-success tasks and pickup regression show where this shift stops helping: contact conditions and incomplete procedures still require physical verification. The useful unit of improvement is therefore an operation together with evidence about its applicability.

Acquiring and applying experience are distinct capabilities. Astra-produced experience helps Sol and Luna where their own exploration yields little or no improvement. An agent can thus use knowledge that it struggles to acquire, allowing one producer's exploration to support multiple executors. The cross-task results further suggest that reuse depends on the contents of an artifact: the selected related-task mapping underperforms the mismatched mapping. Shared motion checks and recovery guidance may transfer across different task labels. This motivates selecting experience by the operations and conditions a receiver needs, with applicability checked against its current observations.

Future directions. Our evaluation on RoboDojo-Sim provides a starting point for testing harness evolution across additional benchmarks, robot platforms, and task distributions. Long agent-call latency also motivates more efficient execution: future work can explore reusing validated routines with fewer model calls and allocating intensive reasoning to uncertain states or recovery. Evaluating elapsed task time alongside success would clarify the resulting responsiveness–reliability tradeoff.

6 Conclusion

We presented EmbodiedEvo, a framework for coding agents to turn physical interaction into reusable Skills and Lessons without updating model parameters. On RoboDojo-Sim, EmbodiedEvo achieves 44.48 Average Score and 37.82% SR. Paired evaluations show consistent gains across different input settings. Skills and Lessons achieve comparable observed performance to raw interaction history at 53.0% lower Test execution cost. These findings suggest that harness evolution improves the starting point of task solving by preserving executable operations and the conditions for applying them. Precise contact and reliable long-horizon execution remain important directions for extending this approach.

References

[1] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL https://arxiv.org/abs/2307.15818.

[3] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL https://arxiv.org/abs/2504.16054.

[4] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. URL https://arxiv.org/abs/2209.07753.

[5] Letian Fu, Justin Yu, Karim El-Refai, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026. URL https://arxiv.org/abs/2603.22435.

[6] Zeyu Shen, Haoxiang You, Yilang Liu, et al. EmbodiedSWE: Coding agents for long horizon dexterous robotics. *arXiv preprint arXiv:2609.27308*, 2026. URL https://arxiv.org/abs/2609.27308.

[7] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, et al. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026. URL https://arxiv.org/abs/2609.24170.

[8] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904*, 2026. URL https://arxiv.org/abs/2605.23904.

[9] Seth Karten, Joel Zhang, Tersoo Upaa, Jr., Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual Harness: Online Adaptation for Self-Improving Foundation Agents. *arXiv preprint arXiv:2605.09998*, 2026. URL https://arxiv.org/abs/2605.09998.

[10] Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery. *arXiv preprint arXiv:2604.01658*, 2026. URL https://arxiv.org/abs/2604.01658.

[11] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. *arXiv preprint arXiv:2407.08693*, 2024. URL https://arxiv.org/abs/2407.08693.

[12] Physical Intelligence, Ali Amin, Raichelle Aniceto, et al. $\pi^*_{0.6}$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025. URL https://arxiv.org/abs/2511.14759.

[13] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, et al. Transferring the intelligence of VLMs to robotic control. *arXiv preprint arXiv:2609.22966*, 2026. URL https://arxiv.org/abs/2609.22966.

[14] Yuyao Liu, Jiayuan Mao, David Hsu, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. RAPID: Robot agentic programming from demonstrations. *arXiv preprint arXiv:2609.30249*, 2026. URL https://arxiv.org/abs/2609.30249.

[15] So Kuroki and Yujin Tang. Learning and transferring closed-loop robot software. *arXiv preprint arXiv:2609.19906*, 2026. URL https://arxiv.org/abs/2609.19906.

[16] Hongcheng Gao, Jingjing Zhou, Zelin Zheng, Shijia Ge, Jay Zhu, Yazhe Wang, Jianshu Zeng, Xuan Shangguan, Di Wu, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. Self-evolving coding agents: From digital programs to physical-world intelligence. *arXiv preprint arXiv:2609.35432*, 2026. URL https://arxiv.org/abs/2609.35432.

[17] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-Evolving Reasoning LLM from Zero Data. *arXiv preprint arXiv:2508.05004*, 2025. URL https://arxiv.org/abs/2508.05004.

[18] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*, 2026. URL https://arxiv.org/abs/2602.08234.

[19] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-Evolution of Agent Memory Systems. *arXiv preprint arXiv:2512.18746*, 2025. URL https://arxiv.org/abs/2512.18746.

[20] Peidong Wang, Zhiming Ma, Ying Chang, et al. Self-evolving embodied agents via skill-harness evolution. *arXiv preprint arXiv:2608.11350*, 2026. URL https://arxiv.org/abs/2608.11350.

[21] Kailin Wang, Haoxiang Jie, Yaoyuan Yan, Zhiyou Heng, and Zhaosong Li. RegenHarness: A robot agent harness with evidence-gated recursive self-improvement. *arXiv preprint arXiv:2609.27612*, 2026. URL https://arxiv.org/abs/2609.27612.

[22] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026. URL https://arxiv.org/abs/2603.19461.

[23] Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening Static Worlds for Agent Learning. *arXiv preprint arXiv:2608.19880*, 2026. URL https://arxiv.org/abs/2608.19880.

[24] Junyi Zhang, Jiaxin Ge, Hanjun Yoo, et al. Playful agentic robot learning. *arXiv preprint arXiv:2606.19419*, 2026. URL https://arxiv.org/abs/2606.19419.

[25] Sicheng Xie, Yitong Chen, Haidong Cao, Shunlin Lu, Zuxuan Wu, and Yu-Gang Jiang. Explore, execute, evolve: A skill

acquisition and reuse loop for embodied agents. *arXiv preprint arXiv:2609.37810*, 2026. URL https://arxiv.org/abs/2609.37810.

[26] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026. URL https://arxiv.org/abs/2607.00272.

[27] Shifeng Bao, Fanding Huang, Yihan Lin, et al. RoboHarn-Evo: Evolving hierarchical physical knowledge for self-improving robotic manipulation. *arXiv preprint arXiv:2609.37583*, 2026. URL https://arxiv.org/abs/2609.37583.

[28] Python Song, Zhixuan Liang, Kelsey Fu, Mengdi Wang, Junfeng Yang, and Shilong Liu. EmbodiedRSI: Active continual robot learning through hypothesis-guided co-evolution. *arXiv preprint arXiv:2610.10498*, 2026. URL https://arxiv.org/abs/2610.10498.

[29] HKU MMLab and KAI. PhysicalRSI 1.0: Recursive self-harness for scaling embodied skills. Project page, 2026. URL https://mmlab.hk/research/PhysicalRSI. Accessed October 12, 2026.

[30] RoboProbe. RoboProbe: An agentic robot manipulation codebase. GitHub repository, 2026. URL https://github.com/RoboProbe/RoboProbe.

[31] Tianxing Chen, Yue Chen, Zixuan Li, et al. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026. URL https://arxiv.org/abs/2607.04434.

[32] RoboProbe. GPT-6 Astra per-layout RoboDojo evaluation records. Public repository, revision `9ffacc54f372`, 2026. URL https://github.com/RoboProbe/RoboProbe/blob/9ffacc54f372beef6479d61711bf4945e31ae4ce/results/l3_inspect_eef_official_2100/astra_task_layouts.json.

[33] RoboProbe. RoboDojo task recipes. Repository snapshot, revision `523007520803`, 2026. URL https://github.com/RoboProbe/RoboProbe/tree/523007520803250860958 54e01977a8cf281b684/policy/RoboDojo_Agent_L3_Inspect/recipes. Fixed task descriptions, public scoring conditions, and upstream task notes used as experiment inputs.

A Playground and Test Instructions

Playground instruction

```
# Goal
You are an embodied coding agent controlling a robot through code. You are currently in
Playground Phase. Your goal is to explore the current robot task, develop a working
solution, and extract reusable skills and lessons that may transfer to future scenes.
[...]
Organize your work as an iterative loop:
observe -> reason -> implement -> record reusable knowledge when it appears -> submit
-> inspect -> revise
[...]
Whenever an observation reveals a reusable controller pattern, a repeated failure, a
recovery strategy, or an important edge case, pause and write it to `skills/` or
`lessons/` immediately while the evidence is fresh. Do not wait until the end of the
session to record it.
# Environment and Primitives
Read `primitives/primitives.md` before interacting with the environment. You have four
native primitives:
- `get_instruction()`: read the current task and goal.
- `get_observation()`: read the latest observation, including camera images and robot
state.
- `step(action, ...)`: execute a native controller action.
- `reset()`: restore the current scene and renew its official native action allowance.
The total {{EXECUTION_BUDGET}} code-execution requests and elapsed time are never reset.
[...]
# Evolution: skills and lessons
[...]
A skill should capture a general behavior or procedure, not merely rename a sequence of
open-loop `step` calls. A useful skill should:
- be parameterized by relevant targets, tolerances, and control budgets;
- derive scene-dependent decisions from observations and explicit inputs;
- be reusable across similar tasks or scenes;
- specify its preconditions and limitations;
- include evidence from experiments that supports its behavior.
Lessons are textual knowledge about what worked, what failed, and how to recover.
Record useful:
- failure patterns;
- diagnostic signals;
- recovery strategies;
- important edge cases;
- unresolved questions.
[...]
# Playground rules
You may reset with `reset()` in code or `python scripts/env.py reset` and update
`skills/` and `lessons/`. Exploration uses the configured single scene. Success here is
evidence about this scene; transfer to unseen scenes remains unverified.
Keep the task, primitives, and success conditions unchanged. Your transferable
deliverables are the skills and lessons you develop.
You cannot use `reset()` in formal evaluation.
[...]
When done, out of slots, or unable to continue, review the current `skills/` and
`lessons/` and run `finish`. If the connection breaks, query `status` before
resubmitting.
```

Test instruction

```
# Goal
You are an embodied coding agent controlling a robot through code. You are currently in
Test Phase. Your goal is to complete the current robot task using the available
primitives and the skills and lessons from Playground.
[...]
Organize your work as an iterative loop:
observe -> reason -> implement -> submit -> inspect -> revise
[...]
# Environment and Primitives
Read `primitives/primitives.md` before interacting with the environment. You have three
native primitives:
- `get_instruction()`: read the current task and goal.
- `get_observation()`: read the latest observation, including camera images and robot
state.
- `step(action, ...)`: execute a native controller action.
[...]
# Using skills and lessons
[...]
When using a skill:
- check its preconditions, limitations, and experimental evidence against the current
observations;
- choose targets, tolerances, and control budgets appropriate to the current task;
- derive scene-dependent decisions from live observations and explicit inputs;
- inspect feedback after execution and decide the next stage from the resulting robot
state.
Use lessons to recognize failure patterns, interpret diagnostic signals, and choose
recovery strategies. Distinguish verified findings, scene-specific observations, and
untested hypotheses; a result from Playground may not transfer to this scene.
[...]
# Test rules
Work within a single continuous episode. You may observe, revise code, execute stages,
and recover from failures using the current physical state. Both `reset()` in code and
`python scripts/env.py reset` are forbidden in Test.
Keep the task, primitives, success conditions, skills, and lessons unchanged. Complete
the task using the frozen knowledge and the feedback from this scene.
Each Test scene starts with a fresh agent, workspace, and simulator session. Only
frozen skills and lessons carry over from Playground; its executed program, temporary
files, and observations do not. This scene's code, observations, and analysis do not
carry over to other Test scenes.
[...]
Reset is forbidden in Test. All **{{EXECUTION_BUDGET}} code-execution requests** share
the same **{{NATIVE_ACTION_LIMIT}} native action steps** for this scene; neither
allowance can be restored.
[...]
When the task succeeds, you run out of execution slots, or you cannot continue, save
episode-specific work in `submission/` and `scratch/` and run `finish`. The environment
determines success. If the connection breaks, query `status` before resubmitting.
```

B Per-Task Results

Tables 9 and 10 report complete dev- and full-set Score and SR. The dev table compares Astra and EmbodiedEvo with and without wiki on matched scenes; the full table reports EmbodiedEvo with wiki. Generalization averages standard and random variants equally, and Average weights the five task categories equally.

Table 9 Per-task dev-set results. Each cell is Score / SR (%); all four conditions use Astra/xhigh on matched scenes. Shaded category rows report means; the final row reports the overall Average.

Task	Task description		Task description + wiki	
	Astra	EmbodiedEvo	Astra	EmbodiedEvo
Generalization	**3.19 / 0.00**	**22.36 / 16.67**	**12.33 / 7.64**	**36.15 / 31.25**
arrange_largest_number	3.75 / 0.00	13.75 / 0.00	8.33 / 0.00	49.17 / 41.67
fold_clothes	11.67 / 0.00	13.33 / 0.00	33.33 / 16.67	70.00 / 66.67
hang_mugs	0.00 / 0.00	10.00 / 0.00	0.00 / 0.00	8.75 / 0.00
make_toast	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
pack_objects_into_box	6.67 / 0.00	22.50 / 0.00	6.67 / 0.00	22.50 / 0.00
pour_liquid_into_cup	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	41.67 / 41.67
push_T	0.00 / 0.00	83.33 / 83.33	0.00 / 0.00	83.33 / 83.33
sort_nesting_dolls_by_size	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	25.00 / 25.00
stack_blocks	10.00 / 0.00	100.00 / 100.00	50.42 / 41.67	100.00 / 100.00
stack_bowls	6.25 / 0.00	25.42 / 16.67	40.83 / 33.33	26.67 / 16.67
store_laptop_and_headphones	0.00 / 0.00	0.00 / 0.00	8.33 / 0.00	6.67 / 0.00
sweep_blocks	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Memory	**12.67 / 10.00**	**59.33 / 56.67**	**39.67 / 36.67**	**70.33 / 66.67**
cover_blocks	6.00 / 0.00	86.00 / 80.00	60.00 / 60.00	100.00 / 100.00
imitate_sorting_sequence	10.00 / 0.00	90.00 / 80.00	38.00 / 20.00	62.00 / 40.00
match_and_pick_from_conveyor	60.00 / 60.00	100.00 / 100.00	100.00 / 100.00	100.00 / 100.00
press_by_number	0.00 / 0.00	0.00 / 0.00	20.00 / 20.00	100.00 / 100.00
swap_T	0.00 / 0.00	80.00 / 80.00	20.00 / 20.00	60.00 / 60.00
swap_blocks	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Precision	**11.38 / 5.00**	**29.88 / 15.00**	**15.00 / 7.50**	**23.12 / 12.50**
build_tower	2.00 / 0.00	26.00 / 0.00	6.00 / 0.00	10.00 / 0.00
deposit_coin	8.00 / 0.00	12.00 / 0.00	8.00 / 0.00	36.00 / 20.00
fasten_screws	28.00 / 0.00	70.00 / 40.00	34.00 / 0.00	64.00 / 40.00
insert_key	9.00 / 0.00	15.00 / 0.00	12.00 / 0.00	15.00 / 0.00
insert_tubes	4.00 / 0.00	36.00 / 0.00	0.00 / 0.00	40.00 / 20.00
play_Xylophone	20.00 / 20.00	40.00 / 40.00	40.00 / 40.00	20.00 / 20.00
plug_in_charger	0.00 / 0.00	40.00 / 40.00	0.00 / 0.00	0.00 / 0.00
pour_balls_into_vase	20.00 / 20.00	0.00 / 0.00	20.00 / 20.00	0.00 / 0.00
Long-Horizon	**18.12 / 0.00**	**29.50 / 12.50**	**19.38 / 0.00**	**33.75 / 20.00**
classify_objects	22.00 / 0.00	40.00 / 0.00	30.00 / 0.00	47.00 / 20.00
fill_egg_holder	7.00 / 0.00	7.00 / 0.00	7.00 / 0.00	20.00 / 0.00
fill_pen_holder	15.00 / 0.00	5.00 / 0.00	8.00 / 0.00	2.00 / 0.00
make_kong	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
organize_table	15.00 / 0.00	40.00 / 0.00	40.00 / 0.00	30.00 / 0.00
play_stacking_toy	6.00 / 0.00	4.00 / 0.00	18.00 / 0.00	10.00 / 0.00
play_tic_tac_toe	55.00 / 0.00	100.00 / 100.00	32.00 / 0.00	100.00 / 100.00
put_bottles_into_dustbin	25.00 / 0.00	40.00 / 0.00	20.00 / 0.00	61.00 / 40.00
Open	**28.50 / 27.50**	**52.12 / 50.00**	**17.00 / 15.00**	**48.75 / 45.00**
align_blocks	60.00 / 60.00	80.00 / 80.00	20.00 / 20.00	100.00 / 100.00
classify_objects_by_language	24.00 / 20.00	48.00 / 40.00	16.00 / 0.00	46.00 / 20.00
general_pickup	80.00 / 80.00	60.00 / 60.00	40.00 / 40.00	80.00 / 80.00
pick_from_conveyor_by_image	0.00 / 0.00	40.00 / 40.00	20.00 / 20.00	20.00 / 20.00
pour_by_language	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00	0.00 / 0.00
solve_equation	20.00 / 20.00	100.00 / 100.00	20.00 / 20.00	80.00 / 80.00
stack_blocks_by_language	44.00 / 40.00	84.00 / 80.00	20.00 / 20.00	64.00 / 60.00
store_tools_in_toolbox	0.00 / 0.00	5.00 / 0.00	0.00 / 0.00	0.00 / 0.00
Average	**14.77 / 8.50**	**38.64 / 30.17**	**20.67 / 13.36**	**42.42 / 35.08**

Table 10 Per-task full-set results for EmbodiedEvo (Astra/xhigh). Task description + wiki, from the completed evaluation in Table 3. Shaded category rows report means; the final row reports the overall Average.

Task / category	Score	SR (%)
Generalization	**34.12**	**29.67**
arrange_largest_number	58.20	46.00
fold_clothes	66.80	60.00
hang_mugs	5.40	0.00
make_toast	1.50	0.00
pack_objects_into_box	20.30	4.00
pour_liquid_into_cup	28.00	28.00
push_T	86.00	86.00
sort_nesting_dolls_by_size	8.00	8.00
stack_blocks	71.60	68.00
stack_bowls	54.50	50.00
store_laptop_and_headphones	3.20	0.00
sweep_blocks	6.00	6.00
Memory	**71.90**	**69.67**
cover_blocks	100.00	100.00
imitate_sorting_sequence	71.40	58.00
match_and_pick_from_conveyor	76.00	76.00
press_by_number	98.00	98.00
swap_T	86.00	86.00
swap_blocks	0.00	0.00
Precision	**23.65**	**13.25**
build_tower	8.60	0.00
deposit_coin	37.20	22.00
fasten_screws	60.00	38.00
insert_key	15.00	0.00
insert_tubes	36.40	14.00
play_Xylophone	16.00	16.00
plug_in_charger	4.00	4.00
pour_balls_into_vase	12.00	12.00
Long-Horizon	**37.20**	**23.50**
classify_objects	44.30	22.00
fill_egg_holder	13.80	0.00
fill_pen_holder	2.00	0.00
make_kong	0.00	0.00
organize_table	41.50	2.00
play_stacking_toy	18.80	0.00
play_tic_tac_toe	100.00	100.00
put_bottles_into_dustbin	77.20	64.00
Open	**55.52**	**53.00**
align_blocks	98.00	98.00
classify_objects_by_language	63.40	46.00
general_pickup	86.00	86.00
pick_from_conveyor_by_image	38.00	38.00
pour_by_language	0.40	0.00
solve_equation	76.00	76.00
stack_blocks_by_language	82.40	80.00
store_tools_in_toolbox	0.00	0.00
Average	**44.48**	**37.82**

C Cross-Task Transfer

Table 11 gives the source assignments and outcomes for the twelve targets in Section 4.4. Related assignments pair tasks with similar operations; the mismatched assignment permutes the same source pool. Both maps were fixed before execution and expose 146 files totaling 388,507 bytes, counted once per target package. Only read-only Skills/Lessons are transferred; the instruction, goal, and scene remain those of the target. Neither transfer condition performs target-task exploration.

All conditions use Astra/xhigh on the same target scenes. Generalization variants receive equal weight within each target, then the twelve targets receive equal weight. The table retains absolute outcomes so that the gains over No Experience in Figure 10 can be reconstructed; these subset means are separate from the full-benchmark Average.

Table 11 Cross-task sources and target outcomes. Each outcome is Score / SR (%). Related and mismatched sources are fixed before these transfer runs; B and D reuse the same target-scene results.

Target	Related source	Mismatched source	B		D		Related		Mismatched	
			Score	SR (%)	Score	SR (%)	Score	SR (%)	Score	SR (%)
stack bowls	stack blocks	swap blocks	6.25	0.00	25.42	16.67	25.42	16.67	15.00	0.00
stack blocks	stack bowls	swap T	10.00	0.00	100.00	100.00	100.00	100.00	83.33	83.33
insert tubes	plug in charger	classify objects	4.00	0.00	36.00	0.00	60.00	40.00	4.00	0.00
plug in charger	insert tubes	classify objects by language	0.00	0.00	40.00	40.00	0.00	0.00	0.00	0.00
fill pen holder	fill egg holder	pour liquid into cup	15.00	0.00	5.00	0.00	2.00	0.00	17.00	0.00
fill egg holder	fill pen holder	pour by language	7.00	0.00	7.00	0.00	8.00	0.00	14.00	0.00
swap blocks	swap T	stack bowls	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
swap T	swap blocks	stack blocks	0.00	0.00	80.00	80.00	0.00	0.00	100.00	100.00
classify objects	classify objects by language	insert tubes	22.00	0.00	40.00	0.00	39.00	20.00	30.00	0.00
classify objects by language	classify objects	plug in charger	24.00	20.00	48.00	40.00	24.00	20.00	32.00	20.00
pour liquid into cup	pour by language	fill pen holder	0.00	0.00	0.00	0.00	0.00	0.00	16.67	16.67
pour by language	pour liquid into cup	fill egg holder	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Fixed-target mean			7.35	1.67	31.78	23.06	21.53	16.39	26.00	18.33

Notes: B: No Experience; D: same-task harness. Standard/random variants are averaged equally within each Generalization task, then the twelve target tasks receive equal weight. These subset means are separate from the full benchmark Average. All executors and experience producers are Astra/xhigh. No new Playground is run for either transfer condition.

D Supporting Case Evidence

D.1 Capability Changes

Table 12 summarizes the execution records behind Section 4.3. All cases use Astra/xhigh; paired comparisons use the same scene. The evidence bundle retains the original programs, feedback, file reads, and camera frames. These selected cases illustrate mechanisms, not failure frequencies or isolated causal effects.

Table 12 Execution evidence for the capability analysis. Action counts refer to native simulator actions; layout IDs identify the matched scenes.

Task / layout	Key execution evidence
Tic-tac-toe / 27	Astra first retains a ring at action 367 and ends with Score 50 at the 1,100-action limit. EmbodiedEvo includes `rim_pick.py`, retains a ring at action 38, and succeeds at action 919. First rings and arms differ.
Imitation sorting / 4	Astra moves before the demonstration finishes and terminates at action 256 with Score 0; feedback does not identify the cause. EmbodiedEvo observes the new order through action 455 and succeeds at action 1,585.
Insert key / 44	EmbodiedEvo's receiver closes empty in execution 14 and remains unsecured through execution 21. Donor release reaches the 300-action limit with Score 15. Failure occurs during handover, before insertion.

D.2 Experience Reuse and Its Limits

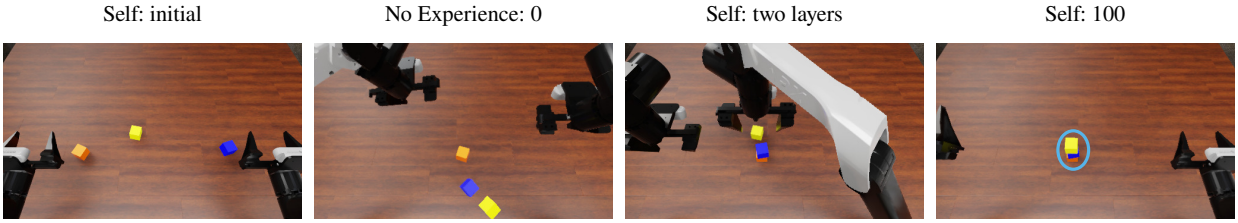
Figure 13 compares successful reuse, cross-model transfer, and a regression. Figure 14 provides the complete retained grasp function and two Lessons used in the successful stacking trace in Appendix F.

Stacking: a correction with a check. The frozen lesson records, "The arm reached its target, but load retention failed." Test execution 2 includes `ee_move.py` and `pick_from_above.py` verbatim. Execution 9 uses an 8 mm translation bound after visual retention checking; execution 15 completes the stack. This establishes actual reuse, without isolating the bound's causal contribution.

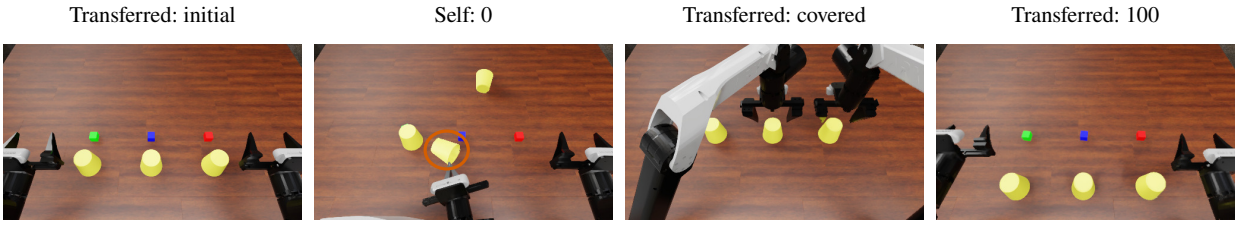
Covering: conditions that transfer. Sol's own artifact describes a "pickup-only experimental skill, not a validated carry or place controller"; its Test loses the cup during transport. Astra's lesson instead distinguishes empty-cup pickup from a higher uncover grasp, validated on three cups. Sol uses these separate heights in executions 2, 11, and 13 and succeeds in execution 17.

Pickup: a transfer boundary. Astra's saved retracting lift succeeds on scissors in Playground. On the elephant, Test execution 7 reports `pose_reached`, but the following observation shows the object was lost. Source-scene success therefore leaves retention on new geometry unresolved; the trace does not isolate the saved advice as the cause of failure.

(a) Language-conditioned stacking · Astra/xhigh · layout 37



(b) Cover and uncover by color · Sol/xhigh · layout 38



(c) Elephant pickup · Astra/xhigh · layout 15

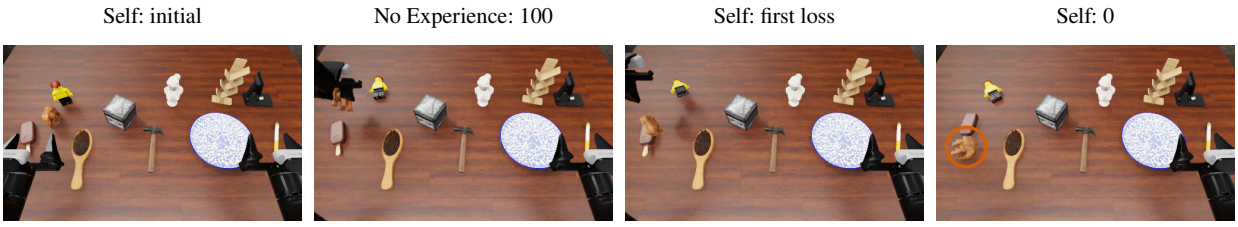


Figure 13 Success, transfer, and regression on matched scenes. (a) Language stacking, Astra/xhigh, layout 37: the self-harness completes the stack. (b) Cover/uncover, Sol/xhigh, layout 38: Astra's harness succeeds where Sol's own harness fails. (c) Elephant pickup, Astra/xhigh, layout 15: Astra succeeds without experience but loses the object with its self-harness. Frames are original observations from independent runs, with identical crops. Labels show official Scores and execution indices.

(a) Skill: a guarded, budgeted grasp function

```
def pick_from_above(arm, grasp_pose, lift_z, action_budget, grip_dwell=12, descend_step=0.010,
  lift_step=0.008, alignment_tolerance=0.003):
    """Pick from an already aligned hover; return for visual retention verification."""
    obs = get_observation()
    current = obs['state'][arm + '_ee_pose']
    lateral_error = sum((current[j] - grasp_pose[j]) ** 2 for j in range(2)) ** 0.5
    if lateral_error > alignment_tolerance or current[2] < grasp_pose[2]:
        return obs, {'error': 'requires_aligned_hover', 'steps': 0}
    used = 0
    stages = [('descend', list(grasp_pose), 1.0, 3, descend_step),
              ('close', list(grasp_pose), 0.0, grip_dwell, descend_step)]
    raised = list(grasp_pose)
    raised[2] = lift_z
    stages.append(('lift', raised, 0.0, 3, lift_step))
    for name, target, grip, dwell, speed in stages:
        remaining = action_budget - used
        if remaining < dwell:
            return obs, {'error': 'budget_exhausted', 'stage': name, 'steps': used}
        obs, result = move_ee(arm, target, grip, max_steps=remaining, min_steps=dwell,
  max_translation_step=speed)
        used += result['steps']
        if not result.get('reached', False):
            return obs, {'stage': name, 'steps': used, 'motion': result}
    return obs, {'stage': 'lifted_needs_visual_check', 'steps': used, 'gripper': obs['state'][arm + '
  _ee_joint_state']}
```

(b) Lesson: verify retention, then carry slowly

A Playground carry dropped the block even though the arm reached its target. The saved rule is to bound each translation and inspect the load after each carry stage. Test execution 9 uses an 8 mm bound after the agent inspects the lifted blue block.

```
obs, result = move_ee('right', base_xy+[1.045]+
  down, 0.0, max_steps=46, max_translation_step
  =0.008)
print(result)
```

(c) Lesson: reserve actions for the terminal check

The saved rule requires home commands after placement and immediate stopping on termination. The stable Test stack still has success=false at execution 14. Execution 15 triggers success after 10 home actions, with five actions left.

```
for i in range(15):
    obs, reward, terminated, truncated, info =
  step(home)
    if terminated or truncated:
        print({'steps':i+1,'reward':reward,'
  terminated':terminated,'truncated':truncated,'
  success':info.get('success',False)})
        break
```

Figure 14 A retained Skill and two Lessons in actual reuse. (a) The complete frozen `pick_from_above` function composes the measured-pose `move_ee` controller into descent, closure, and lifting. It checks alignment and the action budget, then returns for visual retention verification. The same source is included verbatim in Test execution 2 and called for orange, blue, and yellow in executions 4, 8, and 12. (b–c) The lesson descriptions paraphrase the frozen `downward_approach.md`; the highlighted Test snippets are verbatim. The complete episode is shown in Appendix F.

E Raw-History Comparison and Cost Accounting

Matched comparison. Raw History supplies the same Playground's ordered programs and public feedback, including raw observations and failed attempts, through read-only files. It excludes Skills/Lessons and private scores and is not summarized by another model. Both representations use Astra/xhigh on matched Test scenes, with no reset. Skills/Lessons exceed Raw History by 2.23 Score points (95% interval: −2.60–7.25) and 0.39 SR points (−5.11–5.94). Intervals use 10,000 paired task-bootstrap draws within categories (seed 20261006). Similar observed performance is not a formal equivalence result.

Tokens and expense. Figure 11 and Table 8 average terminal API-usage counters over complete Test episodes, including failures and repeated context; Playground acquisition is excluded. Total tokens equal input plus output, with cache reads included in input and reasoning included in output. A missing final event in one local Raw History trace is recovered from its terminal report, without imputation. Cache reads account for 82.6% of Raw History input and 97.6% of Skills/Lessons input. The fixed Astra rates are \$10/\$1/\$12.5/\$50 per million ordinary-input/cache-read/cache-write/output tokens. These nominal rates explain the larger cost saving (53.0%) than total-token saving (11.3%); the amounts are estimates, not invoices.

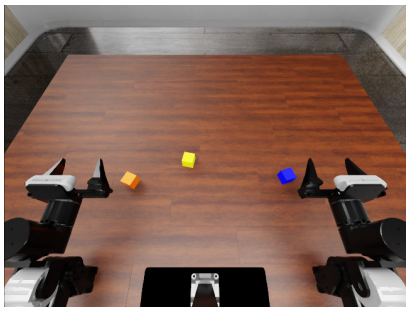
F Complete Successful Agent Trace

This trace covers every accepted execution in the Astra/xhigh self-harness run of `stack_blocks_by_language`, public layout 37. The episode makes 15 execution requests and uses 395 of 400 native actions, ending with official Score 100 and success=true. Each movement stage below pairs a camera image that the agent actually opened *before* the displayed action with its submitted decision comment, highlighted code, and subsequent feedback. Images retain their original field of view. Execution 1 retrieves the instruction alongside the initial image read; executions 2–15 advance the same simulation session without reset.

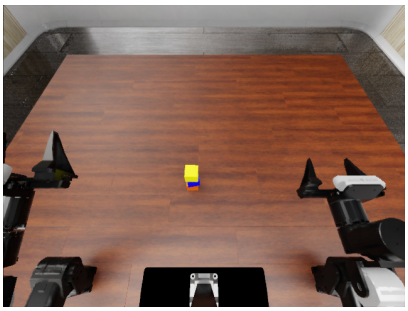
What the agent reads. The agent reads the fixed Test delegation, the native primitive manual, the Skills index, both `ee_move` and `pick_from_above` source files and usage notes, and `lessons/downward_approach.md`. It also reads the initial state metadata and opens the head-camera image. The session records these accesses at lines 18, 25, and 32. The frozen source is prepended to execution 2; later calls reuse the persistent Python namespace. The second helper is reproduced in Figure 14. Execution 7 adds an episode-local `move_both` helper to reposition the empty arms together.

> **Task returned by** `get_instruction()`: "stack the blocks from bottom to top in the order of orange, blue, and yellow, then reset the robot arm."

Here "reset the robot arm" means commanding the saved starting joints, as execution 15 does with `step(home)`. The Test episode never calls the environment's `reset()` primitive.



Initial observation: opened before control



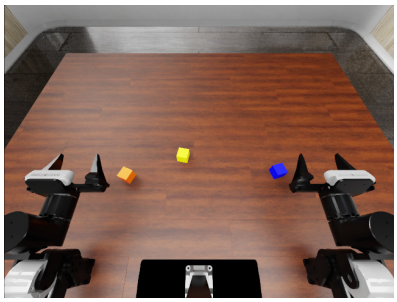
Terminal observation: recorded after execution 15

Figure 15 Endpoints of the complete stacking episode. The initial head view is opened by the agent; the terminal image is recorded by the environment. After execution 15, the agent receives the success feedback and ends without a further image-opening call. The required final order is orange–blue–yellow from bottom to top.

Reading the sequence. Decision text is copied from the agent's submitted code comments; execution 1 has an editorial description of its instruction call. The comments are the recorded rationale for an action, not an independent annotation of what an image proves. Feedback text summarizes the subsequent stdout and public execution counters. The code panels show the complete stage-specific action code except for the repeated imported helpers in execution 2 and the local helper definition in execution 7. The source bundle retains all submitted programs, full stdout, frozen helpers and lessons, and the image-access index in `agent-trace/trace.json`.

Execution sequence

Execution 1: Read the task +0 actions; 0/400 used



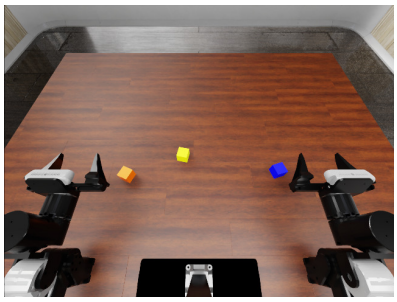
Head camera
Observation 0, during instruction retrieval

Recorded decision. Retrieve the current task instruction before issuing a robot action.

```
print(get_instruction())
```

Feedback. The instruction specifies orange, blue, then yellow; no native action is used.

Execution 2: Load the skills and approach orange +22 actions; 22/400 used



Head camera
Observation 0, before execution 2

Recorded decision. Orange must form the base, with blue and then yellow above it, before returning home. Save home joints and approach orange at a raised pose to calibrate its location safely.

```
obs = get_observation()
home = {k:list(v) for k,v in obs['state'].items() if k.endswith('
  joint_state')}
down = [0.5,-0.5,0.5,0.5]
obs, result = move_ee('left', [-0.20,-0.10,1.10]+down, 1.0,
  max_steps=45)
print(result)
print(obs['state'])
```

Feedback. The raised target is reached in 22 actions; the initial home joint dictionary is saved.

Execution 3: Correct orange alignment

+7 actions; 29/400 used



Left wrist camera
Observation 2, before execution 3

Recorded decision. Orange is left and forward of the wrist's calibrated grasp alignment at this height. Shift the raised left gripper toward it before descending.

```
obs, result = move_ee('left', [-0.225,-0.17,1.10]+down, 1.0,
  max_steps=12)
print(result)
```

Feedback. The raised alignment target is reached in 7 actions.

Execution 4: Grasp and lift orange

+55 actions; 84/400 used



Left wrist camera
Observation 3, before execution 4

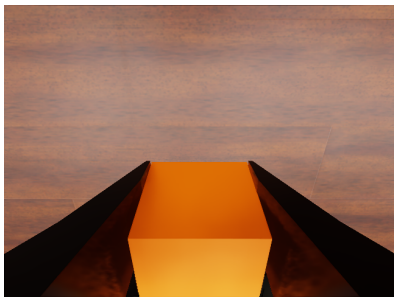
Recorded decision. The raised wrist view localizes orange near (-0.231, -0.188). Align precisely, then use the validated low grasp and slow vertical lift before checking retention.

```
obs, result = move_ee('left', [-0.231,-0.188,1.10]+down, 1.0,
  max_steps=6)
print('alignment', result)
if result.get('reached', False):
    obs, result = pick_from_above('left', [-0.231,-0.188,0.925]+
  down, 1.065, action_budget=52)
    print('pick', result)
print(obs['state']['left_ee_pose'])
```

Feedback. Alignment uses 3 actions and the grasp uses 52. The skill returns lifted_needs_visual_check.

Execution 5: Carry the retained orange block

+28 actions; 112/400 used



Left wrist camera
Observation 4, before execution 5

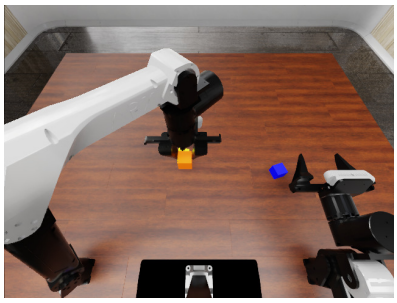
Recorded decision. Orange is visibly retained between the left fingers after lifting. Carry it slowly toward a central base location while keeping clearance above the table.

```
base_xy = [-0.035,-0.188]
obs, result = move_ee('left', base_xy+[1.045]+down, 0.0, max_steps
  =34, max_translation_step=0.008)
print(result)
print(obs['state']['left_ee_pose'])
```

Feedback. The 8 mm translation bound reaches the central base position in 28 actions.

Execution 6: Place the base and retreat

+35 actions; 147/400 used



Head camera
Observation 5, before execution 6

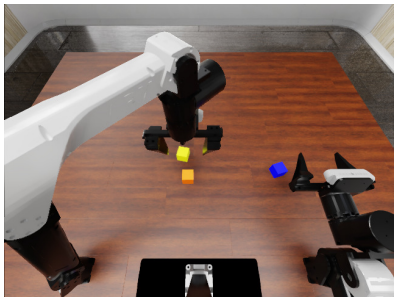
Recorded decision. Orange remains held at the central location with clear table space below. Lower it to table height, release with a dwell, then withdraw vertically.

```
obs, result = move_ee('left', base_xy+[0.928]+down, 0.0, max_steps
  =23, max_translation_step=0.008)
print('lower', result)
if result.get('reached', False):
    obs, result = move_ee('left', base_xy+[0.928]+down, 1.0,
  max_steps=8, min_steps=8)
    print('release', result)
    if not result.get('terminated', False) and not result.get('
  truncated', False):
        obs, result = move_ee('left', base_xy+[1.065]+down, 1.0,
  max_steps=15, max_translation_step=0.018)
        print('retreat', result)
```

Feedback. Descent, release, and retreat take 17, 8, and 10 actions, respectively.

Execution 7: Clear the left arm and approach blue

+12 actions; 159/400 used



Head camera
Observation 6, before execution 7

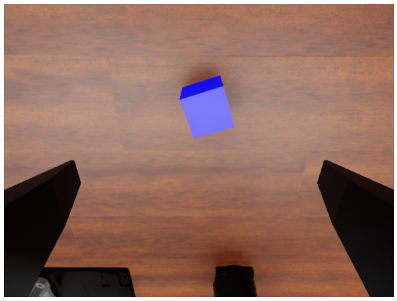
Recorded decision. Orange is stable on the table, and blue is next in the stack. Move the empty left arm aside while the right arm approaches blue at safe clearance.

```
obs, result = move_both([-0.24,-0.24,1.065]+down,
  [0.25,-0.16,1.065]+down, max_steps=22)
print(result)
print(obs['state']['right_ee_pose'])
```

Feedback. A locally defined move_both helper reaches both targets in 12 actions.

Execution 8: Grasp and lift blue

+48 actions; 207/400 used



Right wrist camera
Observation 7, before execution 8

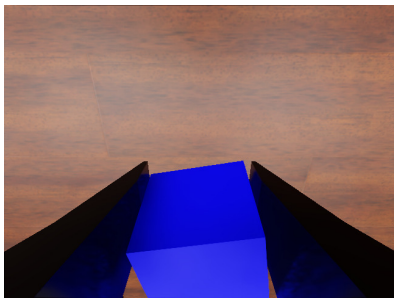
Recorded decision. Blue is slightly right and forward of the calibrated wrist alignment. Correct the raised pose, grasp at table height, and lift slowly to verify retention.

```
blue_xy = [0.2575,-0.166]
obs, result = move_ee('right', blue_xy+[1.065]+down, 1.0, max_steps
  =5)
print('align',result)
if result.get('reached',False):
    obs, result = pick_from_above('right', blue_xy+[0.925]+down,
  1.045, action_budget=48)
    print('pick',result)
```

Feedback. Alignment takes 3 actions and the inherited grasp takes 45; visual verification remains required.

Execution 9: Carry blue above orange

+41 actions; 248/400 used



Right wrist camera
Observation 8, before execution 9

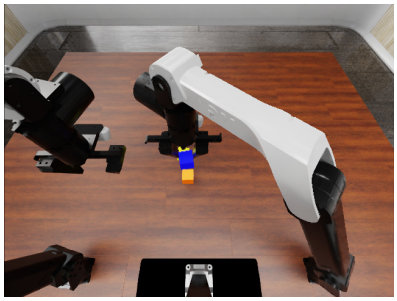
Recorded decision. Blue is visibly retained after lifting, and orange is stable at the central base. Carry blue at the tested loaded speed to the base alignment for a second-layer placement.

```
obs, result = move_ee('right', base_xy+[1.045]+down, 0.0, max_steps
  =46, max_translation_step=0.008)
print(result)
print(obs['state']['right_ee_pose'])
```

Feedback. The loaded carry reaches its target in 41 actions with the 8 mm translation bound.

Execution 10: Place the second layer

+34 actions; 282/400 used



Head camera
Observation 9, before execution 10

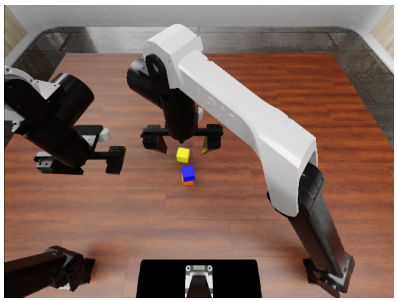
Recorded decision. Blue remains held directly above orange, with the wrist alignment matching the calibrated reference. Lower to the second-layer release height, open, and retreat vertically to inspect the stack.

```
obs, result = move_ee('right', base_xy+[0.968]+down, 0.0, max_steps
  =23, max_translation_step=0.005)
print('lower',result)
if result.get('reached',False):
    obs, result = move_ee('right', base_xy+[0.968]+down, 1.0,
  max_steps=8, min_steps=8)
    print('release',result)
    if not result.get('terminated',False) and not result.get('
  truncated',False):
        obs, result = move_ee('right', base_xy+[1.075]+down, 1.0,
  max_steps=10, max_translation_step=0.018)
        print('retreat',result)
```

Feedback. Descent, release, and retreat take 18, 8, and 8 actions. The next frame shows blue on orange.

Execution 11: Clear the right arm and approach yellow

+15 actions; 297/400 used



Head camera
Observation 10, before execution 11

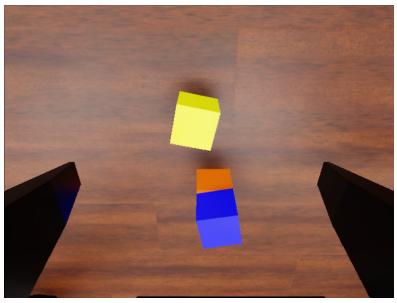
Recorded decision. Blue is stable on orange, leaving yellow as the final layer. Move the empty right arm clear while aligning the left arm above yellow for visual calibration.

```
obs, result = move_both([-0.05,-0.09,1.065]+down,
  [0.24,-0.24,1.075]+down, max_steps=20)
print(result)
```

Feedback. Both raised targets are reached in 15 actions; 103 native actions remain.

Execution 12: Grasp and lift yellow

+50 actions; 347/400 used



Left wrist camera
Observation 11, before execution 12

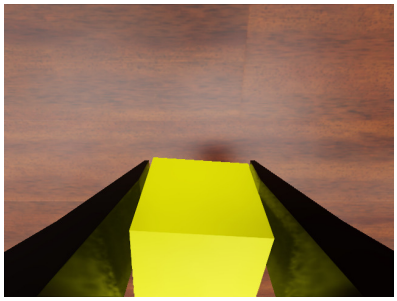
Recorded decision. Yellow is slightly left and forward of the table-height wrist alignment reference. Correct its center, grasp low, and lift vertically with clearance above the two-block stack.

```
yellow_xy = [-0.0535,-0.110]
obs, result = move_ee('left', yellow_xy+[1.065]+down, 1.0,
  max_steps=5)
print('align',result)
if result.get('reached',False):
    obs, result = pick_from_above('left', yellow_xy+[0.925]+down,
  1.045, action_budget=48)
    print('pick',result)
```

Feedback. Alignment takes 3 actions and the grasp takes 47; 53 native actions remain.

Execution 13: Carry yellow over the visible top block

+12 actions; 359/400 used



Left wrist camera
Observation 12, before execution 13

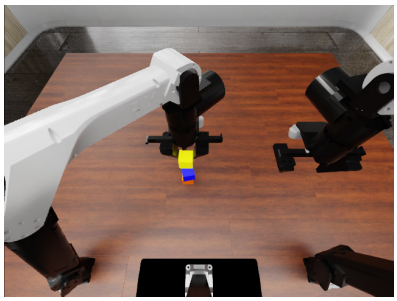
Recorded decision. Yellow is visibly retained, and the orange-blue stack remains upright. Carry yellow over the measured stack center with the tested slow translation limit.

```
top_xy = [-0.033,-0.188]
obs, result = move_ee('left', top_xy+[1.045]+down, 0.0, max_steps
  =15, max_translation_step=0.008)
print(result)
```

Feedback. The corrected stack-center target is reached in 12 actions; 41 actions remain.

Execution 14: Place yellow and preserve a return reserve

+26 actions; 385/400 used



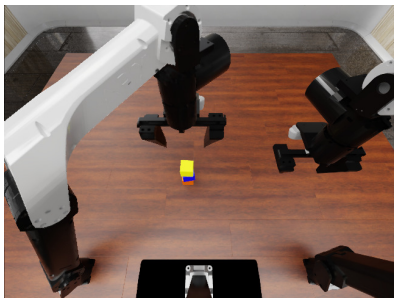
Head camera
Observation 13, before execution 14

Recorded decision. Yellow is held over the actual blue top block, with orange supporting the stack. Place the final layer slowly, release, and retreat before the home-return check.

```
obs, result = move_ee('left', top_xy+[1.008]+down, 0.0, max_steps
  =13, max_translation_step=0.004)
print('lower',result)
if result.get('reached',False):
    obs, result = move_ee('left', top_xy+[1.008]+down, 1.0,
  max_steps=8, min_steps=8)
    print('release',result)
    if not result.get('terminated',False) and not result.get('
  truncated',False):
        obs, result = move_ee('left', top_xy+[1.11]+down, 1.0,
  max_steps=9, max_translation_step=0.018)
        print('retreat',result)
```

Feedback. The 4 mm descent, 8-action release, and retreat use 26 actions. Success is still false; 15 actions remain.

Execution 15: Return home and stop at official success +10 actions; 395/400 used



Head camera
Observation 14, before execution 15

Recorded decision. The camera shows a stable orange-blue-yellow stack after release and vertical withdrawal. Return both arms to the saved starting joints and stop as soon as the official check terminates.

```
for i in range(15):
    obs, reward, terminated, truncated, info = step(home)
    if terminated or truncated:
        print({'steps':i+1,'reward':reward,'terminated':terminated,
  'truncated':truncated,'success':info.get('success',False)})
        break
print(obs['state'])
```

Feedback. Home commands trigger success=true and terminated=true after 10 actions. Five actions remain unused.